天津职业技术师范大学

Tianjin University of Technology and Education

毕 业 设 计

基于 Hadoop+Spark+Hive 的智能出行交通数据可视化分析系统的设计与实现

Design and Implementation of an Intelligent Travel Traffic Data Visualization and Analysis System Based on Hadoop, Spark and Hive

专　　业：　数据科学与大数据技术

班级学号：　大数据 2201 班 - 0307220122

学生姓名：　余天乐

指导教师：　王若凡 副教授

二〇二六年六月

毕业论文（设计）诚信声明

本人郑重声明：所呈交的毕业论文（设计）系在导师指导下独立完成的研究成果。论文中引用他人发表的文献、数据、图表、参考资料等均已通过注释、参考文献等方式明确标注出处。除文中特别标注引用的内容外，本论文不含任何其他个人或集体已公开发表或未公开发表的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。

本人充分知悉并承诺：若本论文涉及知识产权争议或学术不端行为，将由本人承担由此产生的一切法律后果。本声明的法律责任由本人完全承担。

论文（设计）作者签名（手签）：_____

日期：_____年 ____月____日

摘　　要

随着城市出行方式不断丰富，出租车、地铁、共享单车和道路拥堵状态之间呈现明显的时空关联。面向单一数据源的统计系统难以完整描述城市交通运行结构，也难以支撑热点识别、OD 流向分析、拥堵研判和接驳分析等综合应用。围绕天津智能交通场景，设计并实现一套基于 Hadoop、Spark 和 Hive 的智能出行交通数据可视化分析系统，构建从数据采集、清洗、分层存储、批量计算到前端展示的完整技术链路。

系统采用前后端分离架构，后端基于 Django REST Framework 提供统一接口，前端采用 Vue3、TypeScript、Naive UI 和 ECharts 完成交互式可视化展示。大数据侧以 HDFS 承载原始文件，使用 Hive 组织 ODS、DWD、DWS 和 ADS 分层数据，通过 Spark SQL 和 PySpark 完成多源出行明细归一、小时趋势统计、OD 流向聚合、热点区域识别、道路拥堵排名和模型特征构建。系统使用基于真实公开样本分布、天津公开地理事实和交通规律构造的高真实性仿真数据开展实验，主业务数据共 50000 条，并包含出租车轨迹点、天气和地铁拓扑等辅助数据。

测试结果表明，系统能够完成交通数据管理、可视化大屏、热点分析、趋势分析、OD 分析、拥堵分析、模型训练和预测查询等功能，页面展示稳定，接口返回结构清晰，数仓分层能够有效降低前端查询与离线计算之间的耦合度。该设计对城市出行数据教学实验、毕业设计展示和多源交通分析原型验证具有一定参考价值，同时也为后续接入实时流数据、扩展更完整路网和引入更高精度预测模型提供了基础。

关键词：智能出行；Hadoop；Spark；Hive；数据可视化；拥堵预测

ABSTRACT

With the diversification of urban travel modes, taxi trips, subway ridership, shared-bike orders and road congestion records show clear spatial and temporal correlations. A statistical system based on a single data source can hardly describe the overall structure of urban traffic or support integrated applications such as hotspot detection, OD flow analysis, congestion ranking and transfer analysis. This thesis designs and implements an intelligent travel traffic data visualization and analysis system based on Hadoop, Spark and Hive for a Tianjin-oriented traffic scenario.

The system adopts a separated front-end and back-end architecture. The back end uses Django REST Framework to provide unified APIs, while the front end uses Vue3, TypeScript, Naive UI and ECharts for interactive visualization. In the big data layer, HDFS stores raw files, Hive organizes ODS, DWD, DWS and ADS tables, and Spark SQL with PySpark implements unified trip cleaning, hourly trend aggregation, OD flow statistics, hotspot identification, road congestion ranking and model feature construction. The experiment uses high-realism simulated traffic data generated from public sample distributions, Tianjin public geographic facts and common traffic rules.

The results show that the system can support data source management, visualization dashboard, hotspot analysis, trend analysis, OD analysis, congestion analysis, model training and prediction query. The interface structure is clear, the pages run stably, and the layered data warehouse design reduces the coupling between offline computation and online query. The system provides a practical reference for teaching experiments, graduation project demonstration and prototype verification of multi-source urban traffic analysis.

Key Words: intelligent travel; Hadoop; Spark; Hive; data visualization; congestion prediction

目　　录

1 绪论.....1
1.1 研究背景.....2
1.2 研究意义.....4
1.3 国内外研究现状.....6
1.4 研究内容与论文结构.....8
2 相关技术基础.....11
2.1 Hadoop 与 HDFS 技术.....11
2.2 Hive 数据仓库技术.....13
2.3 Spark 计算技术.....15
2.4 Django REST Framework 技术.....17
2.5 Vue3 与 ECharts 技术.....19
2.6 机器学习与预测分析技术.....20
3 系统需求分析.....23
3.1 可行性分析.....23
3.2 用户角色分析.....25
3.3 功能需求分析.....27
3.4 数据需求分析.....30
3.5 非功能需求分析.....33
4 系统总体设计.....36
4.1 系统总体架构设计.....37
4.2 数据处理流程设计.....40
4.3 数据仓库分层设计.....42
4.4 数据库概念结构设计.....46
4.5 主要数据表设计.....48
4.6 接口设计.....51
5 系统详细设计与实现.....54
5.1 数据生成与预处理实现.....54
5.2 Hive 分层表实现.....56
5.3 Spark ETL 任务实现.....60
5.4 后端接口实现.....65

5.5 前端可视化实现.....68
5.6 模型训练与预测实现.....73
6 系统测试与运行结果分析.....76
6.1 测试环境.....76
6.2 数据质量测试.....77
6.3 功能测试.....80
6.4 运行效果展示.....82
6.5 测试结果分析.....87
7 总结与展望.....91

1 绪论

城市交通运行状态直接影响居民通勤效率、城市公共服务能力和道路资源利用水平。随着移动互联网、电子支付、车载定位、地铁闸机和共享单车平台的发展，交通系统在运行过程中产生了大量具有时间、空间和行为属性的数据。这些数据不仅记录了出行需求的变化，也反映了道路拥堵、轨道客流、短途接驳和区域活动强度等城市运行特征。如何把多源异构交通数据转化为可以理解、可以展示、可以辅助分析的信息，是当前交通数据应用中的重要问题。

传统交通管理系统往往围绕固定业务进行建设，例如车辆登记、道路事件上报或某一类交通工具的客流统计。此类系统可以满足单点管理需求，但面对出租车、地铁、共享单车和道路速度等多源数据时，容易出现字段口径不统一、数据链路不完整、分析结果难以复用等问题。大数据技术提供了较为成熟的解决思路：利用Hadoop和HDFS存储大规模原始文件，利用Hive进行分层建模和统一查询，利用Spark完成批量清洗、聚合统计和模型训练，再通过Web系统向用户展示分析结果。

本课题的研究对象为智能出行交通数据可视化分析系统。系统不以单个图表为目标，而是构建一条可复现的数据处理闭环：数据由CSV等文件进入原始层，经清

洗后形成统一出行明细，再汇总为前端可直接读取的专题指标，最终通过地图、折线图、柱状图、排行榜和预测页面展示城市交通规律。该闭环能够体现数据科学与大数据技术专业中数据采集、数据仓库、分布式计算、机器学习和可视化开发之间的衔接关系。

1.1 研究背景

近年来，城市居民的出行方式逐渐从单一公共交通或私家车出行，转向地铁、公交、出租车、网约车、共享单车等多方式组合出行。不同交通方式之间存在明显互补关系，例如地铁承担中长距离骨干通勤，共享单车承担站点周边短距离接驳，出租车在夜间、雨天或跨区域场景中具有更强灵活性，道路拥堵状态又会反向影响车辆出行时长和出行成本。若只观察其中一种交通方式，很难判断城市出行结构的整体变化。

交通数据同时具有规模大、维度多、更新快和空间分布不均衡等特征。以本系统所使用的数据为例，业务明细包含出租车行程、地铁刷卡、共享单车订单和道路拥堵记录，辅助数据包含出租车GPS轨迹点、小时天气、地铁站点和道路维表。不同数据源的字段命名、时间粒度、空间粒度和指标含义并不一致，因此需要先进行

清洗和标准化，再开展跨方式分析。

在实际应用中，交通管理部门、城市运营人员和研究人员更关心的是“哪里拥堵”“哪个时段出行集中”“哪些区域之间流动最强”“站点周边是否存在接驳需求”等问题。这类问题既需要离线计算能力，又需要直观的可视化表达。基于 Hadoop、Hive 和 Spark 构建数仓与计算层，再结合 Django 和 Vue 实现业务系统，是一种适合毕业设计和工程原型验证的技术路线。

城市交通数据应用的发展还受到数据采集条件变化的影响。早期交通信息系统更多依赖固定检测器、人工统计和少量业务报表，数据粒度较粗，更新周期也较长。随着车载定位、移动支付、手机信令和互联网出行平台普及，交通数据的采集对象从单一道路状态扩展到出行个体、换乘行为、区域热度和天气扰动等多个方面。数据维度增加后，简单的报表统计已经难以表达交通系统的复杂关系，需要引入大数据平台和可视化系统进行综合处理。

从城市运行角度看，交通拥堵并不是单一道路速度降低这么简单，它往往与时段、区域功能、天气、节假日、轨道交通供给和短途接驳能力同时相关。例如工作日早高峰的地铁站点周边可能出现单车需求集中，雨天主干路平均速度下降会带动

出租车行程时间增加，节假日期间商圈和交通枢纽周边则可能出现不同于通勤日的出行高峰。因此，论文需要把交通数据放在多源协同分析场景下讨论。

在毕业设计场景中，系统不仅要能够运行，还要能够通过论文材料解释清楚“为什么这样设计”。如果只展示几个页面截图，评阅教师很难判断系统的数据链路是否完整；如果只堆叠技术名称，又无法说明技术和业务需求之间的关系。本文将 Hadoop、Hive、Spark、Django 和 Vue 放在同一条数据流程中说明，使系统从数据产生到结果展示具有连续性。

1.2 研究意义

从工程实现角度看，本课题能够将大数据平台、Web 后端、前端可视化和机器学习模型串联起来，形成完整系统。系统不仅包括页面展示，还包括数据源管理、Spark 任务、Hive 分层表、接口契约和运行测试等内容，能够避免论文停留在静态图表或单个算法实验层面。通过这种方式，毕业设计可以更清晰地展示专业综合能力。

从数据分析角度看，多源交通数据融合能够弥补单一交通方式的局限。出租车数据更适合分析点到点 OD 和道路速度，地铁数据更适合观察站点客流和骨干通勤，

共享单车数据更适合分析短途接驳，道路拥堵数据则用于描述交通运行状态。系统将这些数据统一到时间、区域和出行方式等公共维度上，便于进行横向比较。

从应用展示角度看，可视化系统能够降低数据理解门槛。用户无需直接阅读 Hive 表或 Spark 日志，即可在浏览器中查看综合大屏、热点地图、小时趋势、OD 流向、拥堵排行榜和预测结果。对于答辩和演示场景而言，这种结果呈现方式比命令行输出更直观，也更容易说明系统的设计价值。

本课题的实践意义还体现在对数据科学专业能力的综合训练。交通数据分析涉及数据清洗、字段建模、聚合统计、空间表达、接口开发和可视化设计，这些环节单独看都不复杂，但组合成可运行系统后就需要考虑数据流、模块边界和异常处理。通过完整实现该系统，可以把课程中学习的数据仓库、分布式计算、数据库、Web 开发和机器学习知识整合到一个项目中。

从教学实验角度看，该系统能够作为交通大数据课程或综合实训的样例。教师可以围绕同一套数据讲解 ODS、DWD、DWS 和 ADS 分层，也可以让学生观察 Spark 任务输出如何被后端接口读取，再由前端图表展示。相比单独讲解某个算法或某个页面，这种系统化案例更容易帮助学生理解大数据工程的整体链路。

从工程复用角度看，系统虽然面向毕业设计实现，但其中的数据源登记、任务状态管理、统一接口返回、数仓分层和前端图表组织方式具有一定通用性。后续如果将交通数据替换为停车、公交、校园出行或物流配送数据，只需调整数据源字段和指标口径，整体框架仍然可以复用。

1.3 国内外研究现状

国外在交通数据开放和城市移动性分析方面起步较早。纽约出租车行程数据、MTA 地铁客流数据、PeMS 道路传感器数据等公开数据集，为交通流预测、OD 需求分析和拥堵模式挖掘提供了基础。相关研究常采用 MapReduce、Spark、时序模型和机器学习方法处理大规模交通记录，并通过地图可视化展示空间分布特征。

国内近年来也在交通大数据平台、城市运行监测和智慧交通应用方面开展了大量实践。许多城市建设了综合交通运行监测平台，用于展示道路拥堵指数、公共交通客流和重点区域运行状态。高校毕业设计和课程项目中常见的技术方案包括 Hadoop 离线分析、Spark 批处理、Hive 数仓、ECharts 可视化和基于机器学习的拥堵预测。

已有系统的不足主要体现在三个方面：一是只关注单一数据源，缺少出租车、

地铁、单车和道路状态之间的统一口径；二是只展示图表，缺少从原始数据到数仓分层再到接口输出的完整链路；三是前端展示与后台计算耦合较高，数据口径调整后页面需要大量修改。本文在这些问题基础上设计分层数据模型和统一 API，使系统更符合工程化实现要求。

从研究方法上看，交通大数据系统通常包含三类工作：数据工程、指标建模和可视化表达。数据工程关注数据是否能够被稳定采集、清洗和存储；指标建模关注统计口径是否能够解释交通现象；可视化表达关注结果是否能够被用户快速理解。本文的设计也按照这三类工作展开，因此论文不仅讨论页面效果，也讨论数据层和计算层的实现依据。

现有毕业设计类交通系统中，容易出现两类问题：一类是只用静态 Excel 或少量 CSV 绘图，缺少大数据平台特征；另一类是只堆砌 Hadoop、Spark 等技术名称，却没有说明这些技术如何与业务功能对应。本文通过 ODS、DWD、DWS 和 ADS 分层，把技术组件与实际指标连接起来，使“用了哪些技术”和“解决了什么问题”之间形成对应关系。

近几年交通预测研究大量关注时空特征建模，尤其是利用图结构表达道路网络

或站点网络之间的关联。虽然本文没有实现复杂的图神经网络模型，但在需求和数据设计上保留了道路、站点、区域和 OD 关系等结构化信息，为后续扩展时空预测模型留下基础。毕业设计阶段优先完成可运行系统和传统机器学习模型，是在实现成本与论文完整性之间的折中。

国内智慧交通平台建设更多关注业务场景落地，例如道路拥堵监测、公交客流分析、交通事件上报和运行态势展示。此类系统通常需要接入多个部门或平台的数据，对数据治理、指标口径和可视化呈现有较高要求。本文虽然不涉及真实政务平台接入，但在系统设计中模拟了数据源登记、质量检查、任务调度和专题分析流程，使论文结构更接近真实平台建设逻辑。

在可视化表达方面，交通数据具有强烈空间属性，单纯使用表格和折线图难以表达区域分布。热点地图、OD 流向、道路排名和站点接驳分析能够帮助用户从空间角度理解交通运行状态。本文采用 ECharts 作为主要图表工具，并在页面组织中采用“总览大屏+专题分析”的结构，既便于展示，也便于答辩时按模块讲解。

1.4 研究内容与论文结构

本课题主要完成以下研究内容：第一，整理智能出行交通数据的业务需求，明

确出租车、地铁、共享单车、道路拥堵和天气等数据源的字段口径；第二，设计基于Hadoop、Spark和Hive的数据处理流程，建立ODS、DWD、DWS和ADS分层模型；第三，实现Django后端接口和Vue前端页面，完成多维交通可视化展示；第四，实现拥堵预测与模式分析功能，对模型训练和预测结果进行展示；第五，结合实际运行视频截图完成系统测试与结果分析。

论文共分七章。第一章介绍研究背景、意义和国内外现状；第二章介绍Hadoop、Hive、Spark、Django、Vue和ECharts等关键技术；第三章进行系统需求分析；第四章给出系统总体设计、数据仓库设计和数据库设计；第五章阐述数据处理、后端接口、前端页面和模型模块的详细实现；第六章给出测试环境、测试用例和运行效果；第七章总结全文并提出后续改进方向。

论文写作过程中还需要注意数据边界说明。由于系统采用高真实性仿真数据，结论部分不能写成“天津市真实交通拥堵规律已经被证明”，而应表述为“系统能够在构造数据上完成多源交通分析流程验证”。这种表述既符合项目材料实际情况，也能避免数据真实性被质疑。

本文研究内容还包括对模板格式和论文结构的工程化生成。论文正文、表格、

图片和参考文献均由 JSON 内容和构建脚本生成，这使后续修改标题、章节、表格或文献时可以直接修改数据源文件，而不需要反复手工调整 Word 格式。对于篇幅较长的毕业论文，这种生成方式能够减少格式漂移。

2 相关技术基础

本系统的技术选型围绕“数据存储、离线计算、业务服务、前端展示和模型分析”五个层面展开。大数据侧采用Hadoop、Hive和Spark构建离线分析能力，业务侧采用Django REST Framework提供接口，前端采用Vue3与ECharts实现可视化交互，机器学习模块使用Python生态完成特征处理和模型训练。各技术之间通过清晰的数据文件、数据库表和API契约连接，避免把所有逻辑集中在单一模块中。

2.1 Hadoop与HDFS技术

Hadoop是一套面向大规模数据处理的开源框架，其中HDFS负责分布式文件存储，MapReduce负责批处理计算，YARN负责资源调度。虽然本系统的演示数据规模能够在本地完成处理，但论文设计仍采用HDFS作为原始数据存储层，用于体现大数据系统的数据组织方式。出租车、地铁、单车、道路拥堵和天气等CSV文件可以按主题目录上传到HDFS，后续由Hive外部表或Spark任务读取。

HDFS的优势在于适合存储大文件和历史明细数据。交通数据通常按天、按小时持续产生，单日文件可追加到按日期划分的目录中，便于离线批处理任务统一读取。系统在设计时将原始文件和计算结果分离：原始数据进入ODS目录，清洗后的

统一明细进入DWD层，聚合指标进入DWS或ADS层。这样既保留了数据追溯能力，又能提高前端查询效率。

在本系统中，HDFS并不直接服务前端页面，而是作为离线处理链路的底层数据湖使用。这样设计的原因是，原始交通明细数据在字段和体量上并不适合被前端直接读取，前端更需要的是经过聚合后的指标。HDFS保留全量历史，Hive和Spark完成计算，前端只面向结果层查询，可以避免浏览器承担不必要的数据处理压力。

Hadoop生态适合保存按日期分区的历史文件。若系统后续接入每日新增的出租车行程或道路状态，只需按dt目录追加文件，再由分区表识别新增数据。对于毕业设计而言，这一设计能够说明系统具有持续扩展能力，而不是一次性读取固定样例文件。

HDFS在交通数据系统中的价值不仅是容量大，还在于它能够以目录方式组织历史数据。交通明细通常按日期、数据源和批次保存，例如/taxi/dt=2026-04-01、/subway/dt=2026-04-01等。通过这种组织方式，系统可以保留不同批次的原始文件，便于在数据口径发生变化时重新计算，也便于追踪某个统计结果来源于哪些输入文件。

Hadoop 生态中的 YARN 能够管理计算资源，使 Spark 任务在多节点环境中分配 Executor 并行处理数据。虽然本系统在本地演示时主要采用 local 模式，但论文中仍然保留 YARN 和集群部署说明，是为了体现系统具备向真实大数据环境迁移的可能。若后续数据量从 5 万条扩展到千万级，计算任务可以从单机迁移到集群，而前端和后端接口不需要大幅修改。

在文件格式选择方面，原始 ODS 层可以保留 CSV 以便检查和演示，DWD 和 DWS 层则更适合使用 ORC 或 Parquet 等列式格式。列式存储在读取少量字段和聚合统计时更高效，也能配合 Hive 分区提升查询性能。本文在设计层说明列式存储的原因，有助于体现数据仓库设计不是简单建表，而是考虑了后续查询成本。

2.2 Hive 数据仓库技术

Hive 建立在 Hadoop 之上，提供类 SQL 的数据仓库能力。对于毕业设计系统而言，Hive 的价值不只是查询数据，还在于通过表结构明确数据口径。ODS 表保留原始字段，DWD 表完成清洗和字段统一，DWS 表面向分析主题进行聚合，ADS 表面向页面接口提供轻量结果。分层设计能够将数据处理责任拆开，使每一层的输入和输出更清晰。

交通数据存在多种时间字段和空间字段。例如出租车使用 pickup_time 和 dropoff_time，地铁使用 entry_time 和 exit_time，单车使用 start_time 和 end_time，道路拥堵使用 timestamp。若前端直接访问这些原始表，查询逻辑会非常复杂。DWD 层把这些字段统一为 start_time、end_time、origin_name、destination_name、origin_lng、origin_lat 等公共字段，为后续 Spark 聚合提供统一基础。

Hive 表设计还需要考虑字段类型和分区策略。交通数据中的时间字段应统一转换为 TIMESTAMP 或标准字符串，数值字段应转为 DOUBLE 或 INT，避免后续 Spark 计算时频繁进行类型判断。分区字段一般选择 dt，因为交通数据天然具有日期属性，按天分区既符合数据生成方式，也便于后续按时间范围查询。

数仓分层并不是为了增加复杂度，而是为了降低系统维护成本。如果所有分析都直接基于原始表完成，那么每新增一个图表就可能重复编写清洗逻辑；如果先形成 DWD 公共明细表，再在 DWS 层沉淀主题指标，新增图表时只需要选择合适的汇总表即可，数据口径也更容易保持一致。

Hive 的分层建模需要处理“保留原始信息”和“统一分析口径”之间的矛盾。ODS 层强调不破坏原始字段，便于追溯；DWD 层强调统一字段，便于跨源计算。如果过

早在 ODS 层修改字段，后续难以验证数据来源；如果一直使用原始字段，Spark 聚合逻辑会变得复杂。本文采用 ODS 保留、DWD 统一的方式，较好地平衡了这两个目标。

在交通数据仓库中，维度设计同样重要。时间维度可以拆分为日期、小时、星期、工作日和节假日；空间维度可以拆分为行政区、道路、站点、区域和经纬度网格；交通方式维度包括 taxi、subway、bike 和 road。通过这些公共维度，系统可以把不同来源的记录统一到可比较的坐标系中。

Hive 查询优化还需要考虑小文件问题。若每次任务都输出大量小 CSV 文件，后续查询会产生较高元数据开销。系统设计中将离线结果按主题汇总，并在必要时合并输出文件，能够减少前端查询和后端读取压力。虽然本系统演示规模较小，但这些设计说明可以体现对大数据平台运行机制的理解。

2.3 Spark 计算技术

Spark 是面向大规模数据处理的通用计算引擎，支持 Spark SQL、DataFrame、机器学习和流式处理等能力。与传统 MapReduce 相比，Spark 在迭代计算和交互式分析方面效率更高，适合完成交通数据中的多维聚合、热点统计和模型训练。系统

中的 Spark 任务主要负责从 ODS 或本地 CSV 读取数据，进行字段转换、异常过滤和统计汇总，并将结果写入 DWS 表或本地结果文件。

Spark DataFrame API 能够以声明式方式描述清洗逻辑，例如过滤空值、转换时间类型、统一字段名和按维度分组统计。系统将出租车、地铁和单车数据转换为统一出行明细表，再以 hour_of_day、transport_type、origin_name 和 destination_name 等字段进行聚合，输出小时趋势、OD 流向、热点区域和拥堵排行榜等结果。

Spark 在本系统中同时承担清洗和聚合两类职责。清洗阶段重点处理空值、类型转换和字段映射，聚合阶段重点完成 groupBy 统计、排序和结果输出。由于出租车、地铁和单车数据结构不同，Spark 任务先通过 select 和 alias 构造公共字段，再使用 unionByName 合并，这种实现方式比直接拼接原始字段更稳健。

Spark SQL 的优势在于表达复杂聚合较为直观。例如 OD 流向分析可以理解为按起点、终点和交通方式分组计数，道路拥堵排名可以理解为按道路分组计算平均拥堵指数和最大拥堵指数。通过将这些计算固化为 DWS 输出，前端页面只需要关注展示逻辑，不需要理解复杂的数据处理细节。

Spark 任务的实现需要注意字段对齐。出租车、地铁和单车数据在原始字段上并

不完全一致，直接 union 会导致字段错位或空值异常。因此任务先通过 select、alias 和 cast 构造统一字段，再使用 unionByName 合并。该方法能够减少字段顺序变化带来的风险，也便于后续新增数据源。

在聚合计算中，Spark 的 groupBy、agg 和 window 函数可以处理多数分析需求。例如小时趋势属于按 hour_of_day 分组统计，OD 流向属于按 origin_name 和 destination_name 组合统计，道路拥堵排名属于按 road_name 聚合后排序。通过将这些计算写入独立函数，代码结构更清晰，也便于在论文中逐项说明。

Spark 任务还需要处理执行失败场景。常见失败原因包括输入文件路径不存在、字段名称变化、时间格式无法解析、Hive 连接失败和输出目录已存在。系统通过任务状态和日志记录将这些异常反馈给后端，前端再展示失败原因。这样设计可以避免用户只看到页面空白而不知道后台发生了什么。

2.4 Django REST Framework 技术

Django 是成熟的 Python Web 框架，具备 ORM、路由、中间件和管理后台等能力。Django REST Framework 在 Django 基础上提供序列化、认证、权限和视图集等接口开发能力，适合构建前后端分离系统。本系统后端主要承担三个职责：一是读

取MySQL中的元数据和任务状态；二是读取Spark或ADS层产出的分析结果；三是向前端提供统一JSON格式的可视化接口。

接口设计遵循统一前缀和统一返回结构。基础路径使用/api/v1/，返回结果包含code、message和data字段。通过该约定，前端可以用一致方式处理成功、失败和空数据状态。对于数据源、任务和可视化等模块，后端分别提供列表、详情、创建、预览和专题查询接口，保证系统功能划分清晰。

Django后端在系统中承担“数据结果服务化”的角色。前端不需要知道某个图表数据来自Hive表、本地CSV还是缓存文件，只需要调用统一接口。后端服务层根据配置选择数据读取方式，并把结果转换为统一JSON结构。这样的封装能够隔离底层数据环境变化，降低前后端耦合。

REST接口还需要关注分页、排序和参数校验。数据源列表、任务列表和日志列表都可能随着使用时间增长而变多，因此接口应支持page、page_size、status和keyword等查询参数。可视化接口则应支持date、transport_type、limit和area等参数，使前端能够按条件筛选结果。

在权限控制方面，管理员和分析用户的接口能力应有所不同。管理员可以创建

数据源、触发任务和查看日志，分析用户主要查看图表和预测结果。后端通过角色字段进行判断，前端通过路由守卫和菜单权限减少误操作。即使毕业设计系统权限逻辑较轻量，也应在论文中说明基本控制思路。

2.5 Vue3 与 ECharts 技术

Vue3 是一种轻量化前端框架，采用组合式 API 后能够更方便地组织页面逻辑。系统前端基于 Vue3、TypeScript 和 Naive UI 实现页面结构，包括概览、大屏、数据源、热点、趋势、OD、拥堵、接驳、数仓、模型、预测和用户等导航模块。TypeScript 为接口返回值和图表配置提供类型约束，有助于降低前后端联调中的字段错误。

ECharts 是常用的数据可视化库，支持折线图、柱状图、饼图、地图、热力图、雷达图和排行榜等多种图表。本系统通过 ECharts 展示小时趋势、出行方式对比、OD 排名、道路拥堵 TopN 和模型结果。地图相关页面结合天津区域坐标和热点权重展示空间分布，使用户可以直观看到中心城区、交通枢纽和重点道路的出行强度差异。

Vue3 的组合式 API 适合组织复杂页面状态。综合大屏页面需要同时管理 KPI、

地图、趋势图、排行表和定时刷新状态，如果所有逻辑写在一个函数中会难以维护。通过将接口请求、图表配置和状态管理拆分为组合函数，页面结构更清楚，后续新增图表也更方便。

ECharts 配置需要兼顾可读性和性能。地图热力点、OD 流向和道路排行等图表不宜一次展示过多数据，否则用户难以阅读，浏览器也可能出现渲染压力。因此前端通常展示 TopN 或聚合后的结果，详细明细则通过表格或提示框补充。系统中的 limit 参数就是为了控制图表负载和阅读效果。

可视化页面还要考虑空状态和异常状态。若后端尚未生成 DWS 结果，页面不应直接报错，而应显示“暂无数据”或提示用户先执行分析任务。若接口返回失败，页面应展示错误提示而不是保留旧图表。此类细节虽然不影响算法结果，但会直接影响系统演示效果。

2.6 机器学习与预测分析技术

交通拥堵预测可以视为基于历史道路状态、时间特征和天气特征的监督学习问题。系统以道路拥堵数据为基础，提取 hour_of_day、day_type、weather_level、avg_speed_kmh、precipitation_mm 等特征，对拥堵等级或拥堵指数进行预测。模型

侧支持随机森林等传统机器学习算法，并保留 K-means 聚类用于出行模式或区域热点分群。

随机森林适合处理非线性特征关系，对异常值和多维特征具有较好鲁棒性；K-means 适合在经纬度、热度分值和时间特征上进行无监督聚类，用于辅助解释热点区域或接驳模式。本文不将模型结论扩展为真实交通治理建议，而是将其作为系统功能闭环的一部分，用于展示数据处理结果如何继续服务预测和分析。

表 2-1 系统主要技术选型

层次	技术	作用
数据存储层	HDFS、MySQL	存储原始交通文件、元数据、用户和任务信息
数据仓库层	Hive ODS/DWD/DWS/ADS	组织交通数据分层口径，支撑离线分析和结果查询
计算分析层	Spark SQL、PySpark	完成清洗、聚合、OD 统计、热点分析和模型特征构建
后端服务层	Django REST Framework	提供认证、数据源、任务和可视化接口
前端展示层	Vue3、TypeScript、ECharts	展示综合大屏、趋势、热点、OD 和预测结果
模型分析层	scikit-learn、K-means、随机森林	完成交通模式聚类和道路拥堵预测

机器学习模块在本文中定位为分析闭环的补充，而不是论文唯一核心。系统首先通过数仓和 Spark 形成稳定指标，再将道路速度、天气、时段和拥堵等级等字段作为模型输入。这样的顺序符合实际数据分析流程：先保证数据可用，再讨论模型效果。

随机森林模型的优点是可解释性相对较好。通过特征重要性可以观察hour_of_day、weather_level、avg_speed_kmh和precipitation_mm等字段对预测结果的影响。前端模型页面可以展示这些指标，帮助用户理解模型不是黑箱输出，而是基于已有交通特征进行判断。

K-means聚类在系统中主要用于辅助热点分析。通过经纬度和热度权重聚类，可以把大量点位归纳为若干热点区域，便于地图展示和文字解释。聚类结果并不直接代表行政区划，而是表示数据层面的空间集中趋势，因此论文需要区分“聚类热点”和“真实交通管理分区”的概念。

3 系统需求分析

需求分析的目标是明确系统需要解决什么问题、面向哪些用户、处理哪些数据以及需要达到怎样的性能和可用性要求。本系统不是单纯的数据展示页面，而是围绕交通数据生命周期展开，覆盖数据导入、数据清洗、分层存储、离线计算、结果查询、可视化展示和模型预测等流程。因此，需求分析既包括用户可见的功能，也包括后台数据链路和非功能要求。

3.1 可行性分析

从技术可行性看，本系统所需技术均为成熟开源技术。Hadoop、Hive 和 Spark 可以构建离线数仓和批处理链路，Django REST Framework 能够快速实现后端接口，Vue3 和 ECharts 能够满足可视化页面开发需求。项目仓库中已包含数据生成脚本、Hive 建表 SQL、Spark ETL 任务、Django 后端、Vue 前端和模型训练模块，具备进一步整合为毕业设计系统的基础。

从数据可行性看，系统采用“真实公开样本分布参照+天津公开地理事实约束+交通规律扰动”的方式构造高真实性仿真数据。该数据不声明为天津官方实测轨迹，但能够用于系统演示、算法验证和论文分析。主业务数据共 50000 条，既能体现大数

据处理流程，又不会造成演示环境部署压力过高。

从经济和实施可行性看，系统可在本地开发机完成主要功能演示，也可扩展到 Hadoop、Hive 和 Spark 环境中运行。对于毕业设计而言，系统可采用本地 CSV 或 ADS 结果文件作为回退机制，保证即使在 Hadoop 集群不可用时，也能展示前端页面和核心分析结果。

可行性分析还需要考虑部署复杂度。完整的大数据环境涉及 Hadoop、Hive、Spark、MySQL、Django 和 Vue 多个组件，如果一次性要求所有组件都在答辩现场稳定运行，风险较高。因此系统采用“完整链路设计+本地结果回退”的策略：论文中说明 HDFS 和 Hive 设计，实际演示可通过已生成结果文件保证页面稳定。

从人员和时间可行性看，毕业设计周期有限，系统实现需要在功能完整性和算法深度之间取舍。本文优先保证数据源管理、数仓分层、ETL 计算、接口服务和可视化页面完整，再实现基础拥堵预测。这样的安排能够让论文结构闭环，不会因追求复杂模型而忽略系统工程实现。

从数据伦理和合规角度看，真实交通轨迹数据可能涉及位置隐私和平台权限问题。本文使用公开样本分布和天津公开地理事实构造高真实性仿真数据，既能体现

交通规律，又避免直接使用敏感个人轨迹。论文在数据需求和结论部分明确这一点，有助于提高表述严谨性。

3.2 用户角色分析

系统主要面向两类用户：管理员和分析用户。管理员负责数据源维护、用户管理、分析任务配置和系统状态检查；分析用户主要使用综合大屏、专题分析和预测页面查看结果。两类用户的功能边界并非完全隔离，但在论文设计中需要明确权限差异，以便体现系统管理需求。

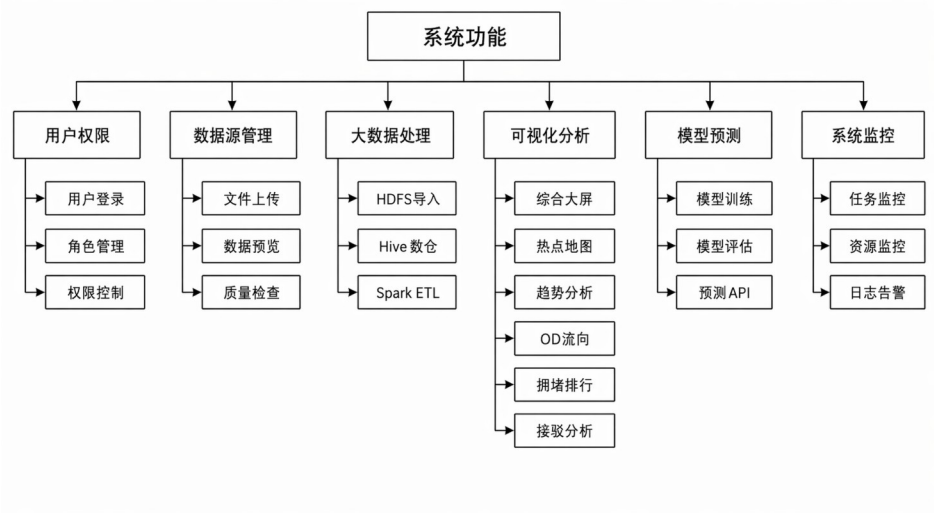

图 3-1 系统功能结构图

如图 3-1 所示，系统功能围绕用户权限、数据源管理、大数据处理、可视化分析、模型预测和系统监控六个模块展开。与普通信息管理系统相比，本系统的特点是把 HDFS 导入、Hive 数仓和 Spark ETL 纳入功能结构中，使用户操作能够与后台

数据链路对应起来。

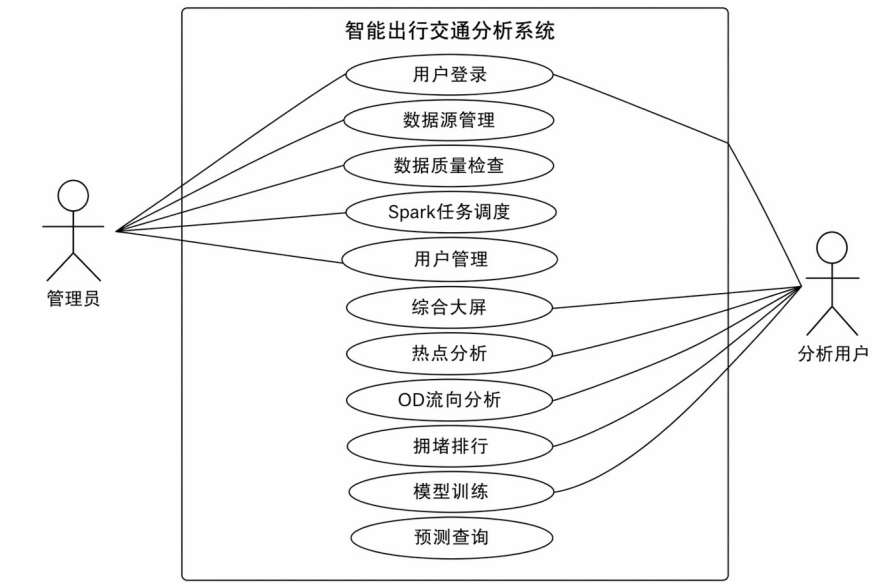

图 3-2 系统总体用例图

如图 3-2 所示，管理员可以进行数据源管理、数据质量检查、Spark 任务调度和用户管理，也可以查看可视化结果；分析用户主要关注仪表盘、热点分析、OD 流向、拥堵排名、模型训练结果和预测查询。通过角色划分，系统既能满足后台维护需求，又能面向普通用户提供简洁的分析入口。

管理员角色的核心需求是保证数据链路可维护。管理员需要知道哪些数据源已经上传、哪些任务已经完成、哪些任务失败以及失败原因是什么。因此，系统不能只提供图表页面，还需要提供数据源、任务和日志管理入口。该入口能够体现系统后台管理能力。

分析用户角色的核心需求是快速理解结果。分析用户通常不关心 Spark 任务内

部如何执行，而是关心某个时段出行量是否异常、某些区域是否形成热点、道路拥堵排名是否变化以及预测结果是否合理。因此，前端页面需要以主题方式组织，而不是把所有字段原样展示给用户。

角色划分还能帮助答辩讲解系统边界。演示时可以先以管理员身份展示数据源和任务，再切换到分析视角展示大屏和专题图表。通过这种顺序，评阅教师更容易理解系统从后台数据到前台展示的完整流程。

3.3 功能需求分析

系统功能需求可分为六个模块。第一是用户认证模块，完成登录、退出和用户信息获取；第二是数据源管理模块，完成 CSV 文件登记、预览、导入和状态展示；第三是大数据分析任务模块，完成 Spark 任务配置、执行、日志查看和结果管理；第四是可视化展示模块，提供大屏、热点、趋势、OD、拥堵和接驳分析；第五是模型分析模块，支持训练、评估和预测；第六是系统管理模块，用于查看依赖状态和基础配置。

表 3-1 系统功能需求表

模块	主要功能	说明
用户认证	登录、退出、用户信息	为系统访问和接口调用提供身份基础
数据源管理	数据登记、上传、预览、字段识别	统一管理出租车、地铁、单车、道路和天气数据

任务调度	Spark任务创建、运行状态、日志查看	管理离线清洗、聚合、模型训练等后台任务
可视化分析	大屏、热力、趋势、OD、拥堵、接驳	向用户展示多维交通运行结果
数仓管理	ODS/DWD/DWS/ADS表说明与结果查看	体现大数据分层处理过程
模型预测	特征选择、模型训练、预测查询	基于拥堵和天气等特征输出预测结果

功能需求的核心在于数据链路和页面展示需要相互对应。例如前端的小时趋势图不能直接依赖出租车、地铁和单车三个原始表，而应读取DWS层统一聚合后的小时出行量；OD流向图也不应分别编写三套逻辑，而应依赖统一的origin_name、destination_name和transport_type字段。这样可以减少前端复杂度，并使数据口径更容易维护。

数据源管理模块还需要支持状态追踪。数据文件登记后并不一定立即可用于分析，可能经历uploaded、checking、ready、failed等状态。状态字段能够帮助用户判断文件是否已经完成校验，也能帮助后台任务选择可用数据源。对于答辩演示而言，数据源页面可以清晰说明系统不是单纯展示静态图表，而是具有数据管理入口。

分析任务模块是连接前台操作和后台计算的关键。用户创建任务后，后端记录任务参数、运行状态和日志路径；Spark任务完成后，将结果文件路径或Hive表名写回数据库。这样即使任务运行时间较长，前端也可以通过轮询或刷新查看状态，

而不需要保持一个长时间阻塞的请求。

可视化模块的需求不应只写“展示图表”，还应明确每张图表的数据来源和业务含义。例如小时趋势图来源于 dws_hourly_trip_count，用于观察日内峰谷；OD 排名来源于 dws_od_flow，用于观察区域联系；拥堵榜单来源于 dws_road_congestion_rank，用于观察重点道路；热力图来源于 dws_hotspot_zone，用于观察空间热点。

数据质量检查属于功能需求中的隐性重点。交通数据只要出现时间格式错误、坐标越界、距离为负或道路名称为空，就可能影响后续图表结果。系统在导入阶段应对关键字段进行检查，并将检查结果写入 quality_flag 或日志中。这样前端展示结果时，才能说明数据已经经过基本校验。

任务调度功能需要支持不同任务类型。ETL 任务负责从原始数据生成 DWD 和 DWS 表，热点任务负责生成地图热力点，OD 任务负责生成起终点流向，模型训练任务负责生成预测模型。将这些任务类型拆开，能够避免一个大脚本承担所有逻辑，也便于前端按任务类型展示进度。

可视化功能需要兼顾总览和细节。综合大屏用于展示总出行量、峰值小时、热

点区域和拥堵道路等概览指标，专题页面用于深入查看某一类分析结果。若所有图表都放在大屏页面，信息密度会过高；若只做专题页面，又缺少整体态势入口。因此系统采用两级展示结构。

模型预测功能需要明确输入输出。输入包括道路、小时、天气、历史速度和日期类型等特征，输出包括拥堵等级、拥堵指数或风险提示。前端页面需要展示输入项、预测结果和模型指标，避免只返回一个数字而缺少解释。

3.4 数据需求分析

系统数据来源包括主业务数据和辅助数据。主业务数据用于直接计算交通指标，辅助数据用于增强空间解释和真实性约束。论文中必须统一说明：天津数据为基于真实公开样本分布、天津公开地理事实和交通规律构造的高真实性仿真数据，不能表述为天津官方实测轨迹或真实运营原始记录。

表 3-2 交通数据源基本信息

数据源	记录数	主要用途	关键字段
天津出租车行程	15000	出租车 OD、热点、距离和时长分析	pickup_time、pickup_zone_name、distance_km
天津地铁刷卡 OD	15000	站点客流、线路 OD 和通勤趋势分析	entry_time、entry_station_name、exit_station_name
天津共享单车订单	15000	短途出行、地铁接驳和单车热点分析	start_time、start_area、nearest_end_station
天津道路拥堵记录	5000	道路速度、拥堵等级和排名分析	road_name、avg_speed_kmh、congestion_index

出租车 GPS 轨迹点	50000	轨迹点热力和空间聚类辅助分析	trip_id、timestamp、lng、lat
小时天气数据	744	天气扰动和拥堵预测特征	temperature_c、precipitation_mm、weather_level
地铁线路拓扑	32	站点接驳和线路结构辅助说明	station_name、line_name、distance_km

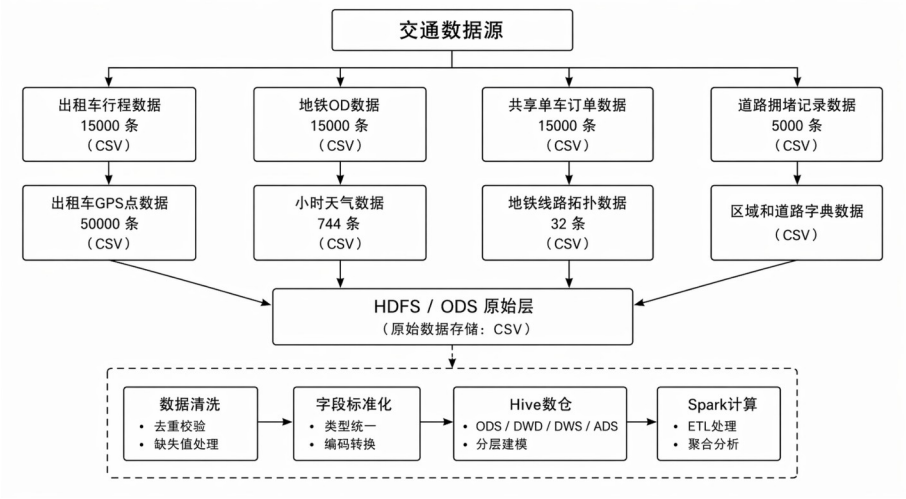

图 3-3 多源交通数据组成图

如图 3-3 所示，系统数据并不是单一表格，而是由出租车、地铁、共享单车、道路拥堵和辅助维表共同构成。主业务表进入 ODS 层后，经过清洗和标准化形成统一分析口径；GPS 轨迹、天气、线路拓扑和区域道路字典等辅助数据则用于增强空间解释、天气扰动和接驳分析。

从字段需求看，系统需要将不同交通方式抽象为统一出行事件。统一事件至少包含记录编号、交通方式、起点名称、终点名称、起点坐标、终点坐标、开始时间、结束时间、出行时长、出行距离、小时、日期类型和天气等级等字段。该抽象是后续跨方式比较的基础。

数据需求还包括数据质量标记。由于系统采用高真实性仿真数据，source_basis、quality_flag和confidence_score等字段能够说明记录生成依据和可信度。论文中保留这些字段，有助于回答数据来源问题，也能避免将仿真数据误写为真实运营数据。

辅助维表对分析解释非常重要。区域维表可以把坐标点映射到天津中心城区、交通枢纽和商圈；地铁站点维表可以支持地铁客流和单车接驳分析；道路维表可以补充道路等级、行政区和自由流速度。没有维表时，明细数据只能进行简单计数，很难形成有解释力的分析结论。

数据字段设计还需要考虑时间粒度。出租车和单车订单具有开始和结束时间，地铁刷卡具有进站和出站时间，道路拥堵记录通常是某一时刻或某一时间段的状态。为了统一分析，系统将所有记录映射到hour_of_day和dt字段上，既可以按小时统计，也可以按日期分区。

空间字段设计需要兼顾名称和坐标。地图展示需要经纬度，表格和排行榜需要区域名称或道路名称。如果只保存名称，无法画热力图；如果只保存坐标，用户难以理解地点含义。因此DWD层同时保存origin_name、destination_name、origin_lng

和 origin_lat 等字段。

交通方式字段是跨源分析的核心。transport_type 字段将出租车、地铁、单车和道路状态区分开，使系统能够在同一张趋势图中对比不同方式，也能够在筛选条件中只查看某一类交通数据。该字段看似简单，但对后续统一接口和前端图例设计非常重要。

天气字段用于解释交通状态变化。降雨、温度和天气等级可能影响道路速度和出行方式选择，因此系统将天气数据作为辅助维度写入分析结果。即使当前模型较简单，保留天气字段也能为后续更复杂预测模型提供基础。

表 3-4 统一出行明细字段需求

字段类别	字段名称	字段说明
标识字段	record_id	统一记录编号，用于追踪来源记录
类型字段	transport_type	交通方式，包含 taxi、subway、bike 等
时间字段	start_time、end_time、hour_of_day、dt	描述出行开始结束时间和分析时间粒度
空间字段	origin_name、destination_name	起点和终点名称，用于 OD 和排行榜展示
空间字段	origin_lng、origin_lat、dest_lng、dest_lat	起终点坐标，用于地图和热力展示
指标字段	duration_min、distance_km	出行时长和距离，用于效率分析
环境字段	weather_level、precipitation_mm	天气等级和降雨量，用于拥堵解释和预测
质量字段	quality_flag、source_dataset	数据质量标记和来源表标记

3.5 非功能需求分析

非功能需求主要包括可维护性、可扩展性、可靠性、可视化可读性和演示稳定

性。毕业设计系统虽然不是生产系统，但仍需要体现工程完整性。系统应能够通过配置切换本地文件和 HDFS 路径，Spark 任务应能在 local 模式和集群模式之间切换，前端页面在接口暂时无数据时也应提供空状态或默认展示，避免运行时出现大面积空白。

表 3-3 系统非功能需求表

需求类型	具体要求	实现方式
可维护性	数据处理、接口和页面模块边界清晰	采用数仓分层、REST 接口和组件化页面
可扩展性	后续可加入公交、实时流或更复杂模型	统一数据源登记和 DWD 公共字段
可靠性	任务失败时可查看日志并保留回退数据	任务状态、日志表和本地 fallback 结果
性能要求	常用图表查询响应较快	DWS/ADS 预聚合，前端不直接扫描明细
安全性	普通用户不能进行高危管理操作	登录认证、角色字段和接口权限控制
展示性	图表布局清晰，适合答辩演示	综合大屏和专题页面分层展示

可靠性要求还包括结果可复现。每次任务运行时应记录输入路径、参数、输出路径和运行时间，后续如果图表结果发生变化，可以回溯是输入数据变化还是任务逻辑变化导致。对于毕业论文而言，可复现性能够增强系统设计可信度。

性能要求需要结合业务场景判断。系统前端页面不需要毫秒级实时刷新，但常用图表查询应在用户可接受时间内返回。因此，系统将复杂计算放在离线 Spark 任务中完成，页面只读取预聚合结果。这种架构比每次请求都扫描明细数据更适合交

通统计场景。

可扩展性要求体现在数据源和指标两方面。新增公交数据时，可以增加 ODS 公交表和 DWD 清洗逻辑；新增停车数据时，可以扩展 transport_type 和相关维表；新增预测模型时，可以在模型结果表中保存新的指标和文件路径。系统不应把数据源写死在前端页面中。

4 系统总体设计

系统总体设计需要回答“数据如何进入系统、如何被处理、如何被保存、如何被查询、如何被展示”这几个问题。本文采用前后端分离和大数据分层相结合的架构：前端通过浏览器访问 Vue 页面，页面调用 Django REST API，后端根据不同接口读取 MySQL 元数据、ADS 结果文件或 Hive 结果表，离线计算任务由 Spark 完成。该架构能够将展示、业务服务和数据计算分开，便于后续扩展。

4.1 系统总体架构设计

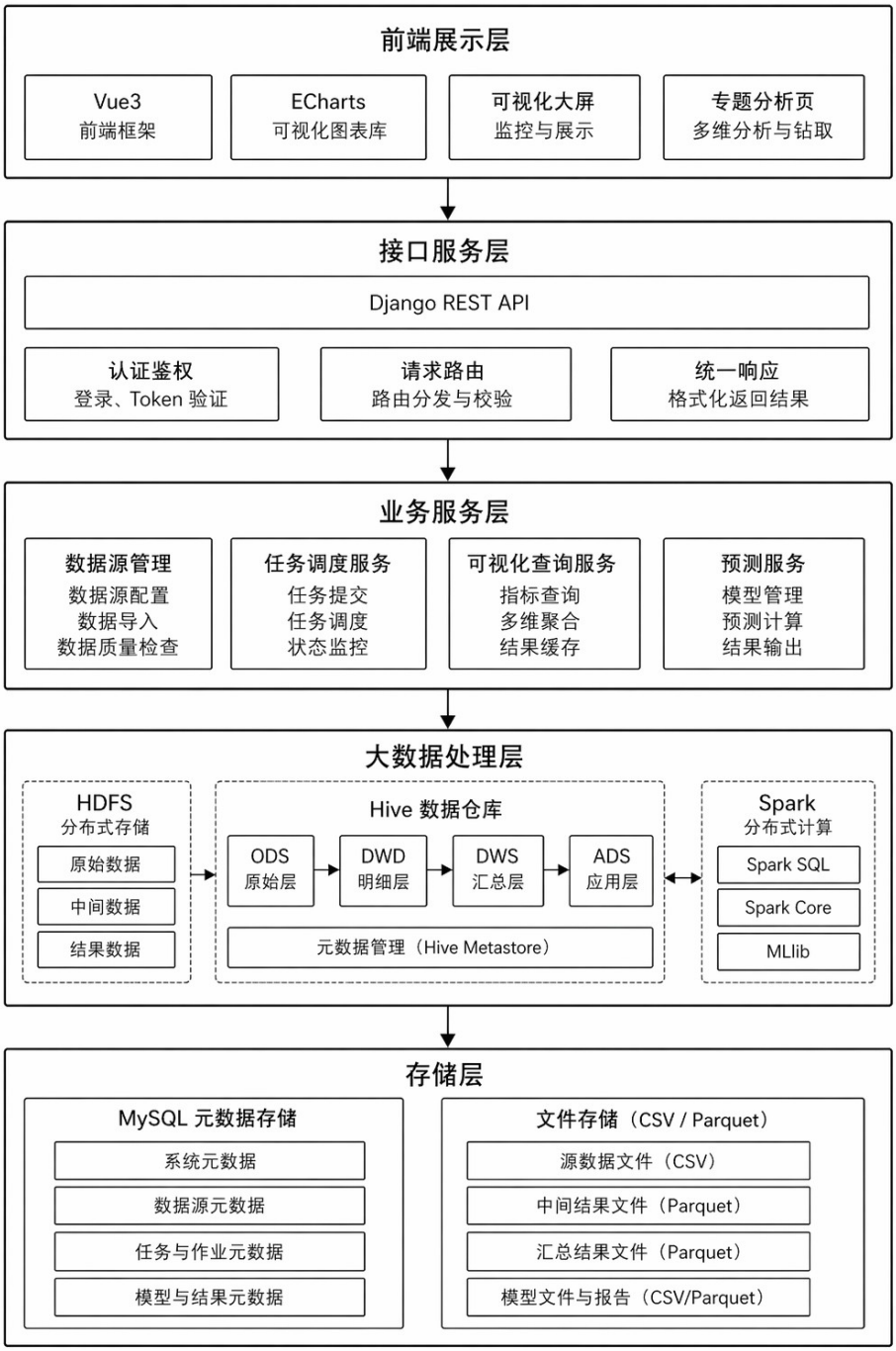

图 4-1 系统总体架构图

如图 4-1 所示，系统由 Web 客户端层、REST 接口层、业务服务层、大数据处理层和存储层组成。Web 客户端层负责页面渲染和图表展示；REST 接口层负责请求

路由、身份认证和返回格式封装；业务服务层负责数据源管理、可视化查询、分析任务调度和预测服务；大数据处理层负责 HDFS、Hive 和 Spark 相关任务；存储层包含 MySQL 元数据和本地或 HDFS 文件。

总体架构的关键是分工明确。前端不直接访问 CSV 或 Hive 表，而是通过接口读取后端整理后的结果；后端不在请求过程中进行大规模聚合，而是优先读取 DWS 或 ADS 层结果；Spark 任务不负责页面展示，只负责离线计算和结果输出。这样可以避免一次页面请求触发大量明细数据计算，提高系统稳定性。

系统总体架构还考虑了演示环境的不确定性。完整环境下，Spark 可以读取 HDFS 文件并写入 Hive 表；本地环境下，Spark 或 pandas fallback 可以读取 data/generated 目录并输出 CSV 结果。后端服务对前端隐藏这些差异，只要输出结构一致，页面就可以正常展示。这种设计提高了毕业设计答辩时的稳定性。

架构中 MySQL 主要保存元数据，而不是承担大规模明细分析。这样划分是因为关系型数据库更适合保存用户、任务、配置和摘要结果，不适合频繁扫描数万乃至更多交通明细。将大规模数据交给 Hive 和 Spark 处理，可以使系统职责更符合各组件特点。

系统架构中各层之间通过明确接口衔接。前端只调用 REST API，不直接读取文件；后端只读取结果表或结果文件，不直接操作页面；Spark 任务只负责数据计算，不关心页面布局；Hive 只负责存储和查询，不承担业务权限控制。这样的分层可以降低系统复杂度，使每个模块职责清楚。

在实际部署时，前端静态资源可以由 Nginx 或 Vite 开发服务器提供，Django 作为后端服务运行，MySQL 保存元数据，Spark 任务通过命令行或后端任务接口触发。若使用集群环境，HDFS 和 Hive 可以部署在独立节点上；若使用本地环境，则用本地目录模拟 HDFS 路径。论文中同时说明这两种模式，能够解释系统为什么既有大数据设计，又能在开发机上演示。

系统总体架构还考虑了数据安全边界。用户密码只保存摘要，后端接口需要登录后访问，上传文件路径和任务日志不直接暴露真实服务器路径。虽然毕业设计系统安全要求不如生产系统严格，但基本安全设计能够体现工程规范性。

4.2 数据处理流程设计

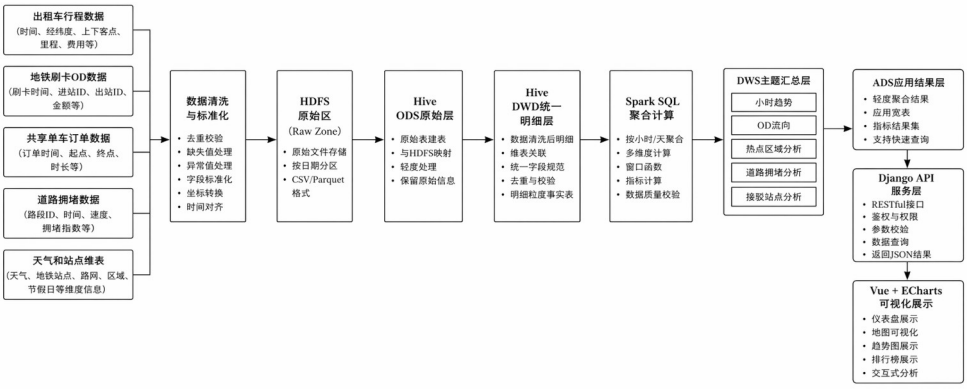

图 4-2 数据流转与分层处理流程图

数据处理流程从原始数据文件开始。出租车、地铁、单车、道路拥堵和天气数据进入原始存储目录后，首先由 ODS 层外部表或 Spark 读取保留原始字段；随后在 DWD 层进行字段统一、类型转换和异常过滤；再由 Spark SQL 按照小时、区域、OD、道路和交通方式等维度汇总为 DWS 层结果；最后由 ADS 层或后端服务面向前端提供轻量化指标。

该流程设计的好处在于，每一层都对应明确的数据质量和业务含义。ODS 层用于追溯源文件，DWD 层用于统一明细口径，DWS 层用于主题聚合，ADS 层用于页面接口。若后续新增公交或停车数据，只需在 ODS 和 DWD 层补充清洗逻辑，再复用 DWS 聚合框架即可。

数据流转过程可以划分为五个阶段：数据登记、原始入湖、明细清洗、主题汇

总和应用服务。数据登记阶段由 MySQL 保存文件信息，原始入湖阶段把 CSV 放入 HDFS 或本地原始目录，明细清洗阶段由 Spark 构造 DWD 表，主题汇总阶段形成 DWS 结果，应用服务阶段由后端接口读取并返回给前端。

在数据登记阶段，系统并不立即假设文件可用，而是先记录文件名称、来源、大小、记录数和状态。只有完成字段检查和基础质量检查后，状态才变为 ready。这样能够避免后续任务读取错误文件，也能在前端页面上向用户说明当前数据是否可用于分析。

在主题汇总阶段，系统针对不同页面生成不同结果。综合大屏需要总体 KPI 和多类图表，趋势页面需要小时维度数据，OD 页面需要起终点流量，拥堵页面需要道路排行，接驳页面需要站点周边短途出行。通过提前汇总，系统可以让页面请求更加轻量。

4.3 数据仓库分层设计

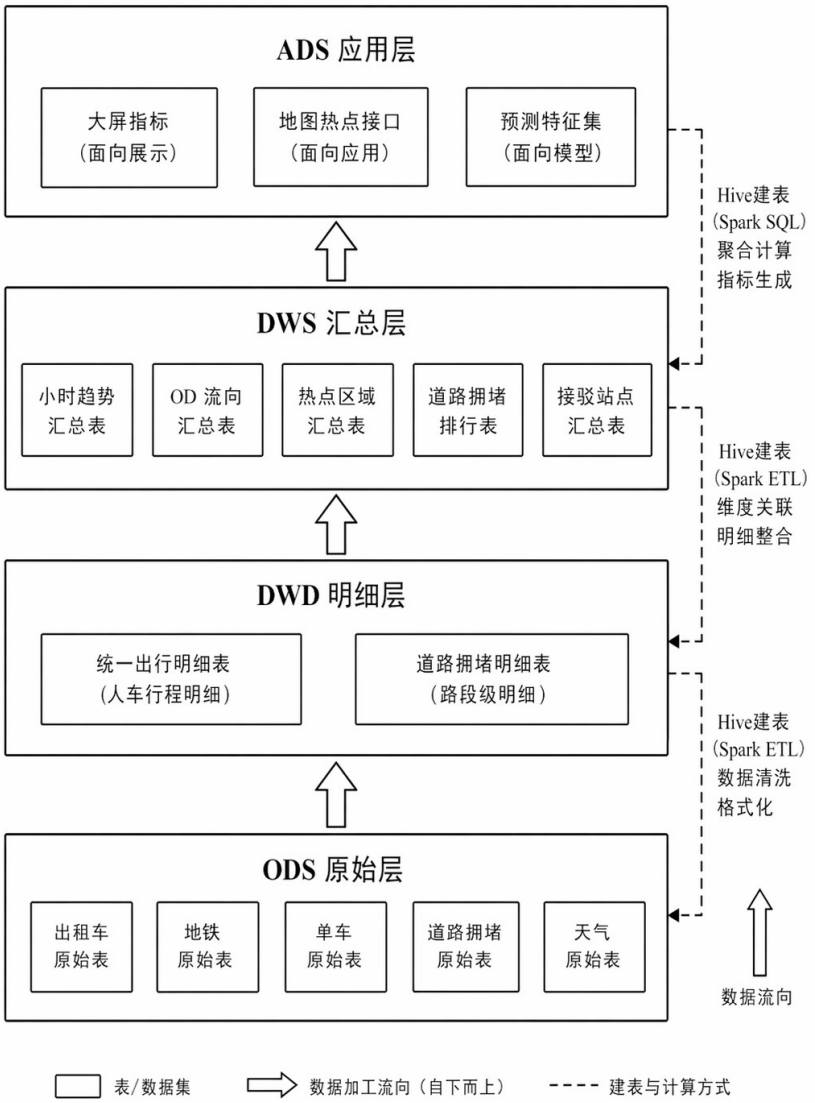

图 4-3 Hive 数仓分层设计图

数仓分层是本系统的设计重点。ODS 层包含 ods_taxi_trip、ods_subway_trip、ods_bike_order、ods_road_congestion、ods_weather_hourly 等表，主要保留源字段和原始文件结构；DWD 层包含 dwd_trip_detail 和 dwd_road_congestion 等表，负责把多源数据统一为可分析明细；DWS 层包含小时出行量、OD 流向、热点区域、拥堵排

名和接驳站点等主题表；ADS 层面向页面和接口输出综合指标。

表 4-1 数仓分层表结构说明

层次	典型表	设计目的
ODS	ods_taxi_trips、ods_subway_trips、ods_bike_orders、ods_road_congestion	映射原始 CSV，保留原始字段和来源信息
DWD	dwd_trip_detail、dwd_road_congestion	统一字段口径，形成明细宽表，支撑跨方式分析
DWS	dws_hourly_trip_count、dws_od_flow、dws_hotspot_zone	按分析主题聚合，减少在线查询成本
DWS	dws_road_congestion_rank、dws_transfer_station	输出拥堵排名和站点接驳强度
ADS	dashboard_metrics、map_hotspot_api、prediction_feature_set	面向页面接口和模型预测提供结果

DWD 层的统一出行明细是系统能否进行综合分析的关键。出租车、地铁和单车虽然在业务上差异较大，但都可以抽象为“某一时间从起点到终点的一次出行事件”。因此，DWD 层保留 transport_type 字段区分交通方式，同时保留公共的起终点、时间、距离、时长、天气和日期类型字段，使后续分析既能整体统计，也能按交通方式拆分。

DWS 层的主题表需要与页面功能一一对应。综合大屏需要 dashboard_metrics，趋势页面需要 dws_hourly_trip_count，OD 页面需要 dws_od_flow，热点页面需要 dws_hotspot_zone，拥堵页面需要 dws_road_congestion_rank，接驳页面需要 dws_transfer_station。通过这种对应关系，可以从页面反推数据来源，也可以从数仓

表反推展示场景。

ADS 层并不一定需要保存大量明细，它更像是面向应用的结果服务层。对于前端页面而言，ADS 层可以将多个 DWS 表组合成一个接口响应，例如综合大屏同时需要 KPI、小时趋势、Top 区域和拥堵排名。提前构造 ADS 结果能够减少前端请求次数，也便于后端缓存。

ODS 层设计强调“少加工”。例如出租车表保留 pickup_time、dropoff_time、pickup_lng、pickup_lat 等原始字段；地铁表保留 entry_station_name、exit_station_name 等字段；单车表保留 start_area、end_area 和 nearest_station 等字段。这样做的目的是保证原始数据可以被重新解释。

DWD 层设计强调“统一口径”。在这一层中，系统不再保留各数据源的全部原始字段，而是抽取共同字段，并补充 transport_type、hour_of_day、day_type 和 weather_level 等分析字段。该层是本文后续所有分析的基础，因此字段命名和类型必须稳定。

DWS 层设计强调“主题明确”。每张 DWS 表对应一个可解释主题，例如小时趋势、OD 流向、热点区域、道路拥堵和接驳强度。主题表不追求保存全部明细，而

是保存页面和模型真正需要的聚合结果。这种设计能够减少接口层数据处理压力。

ADS层设计强调“面向应用”。综合大屏接口可能需要一次返回多个主题的组合结果，因此ADS层可以将DWS表中的数据再次整理为页面友好的结构。对于前端而言，ADS层越贴近页面，页面代码就越简单；对于数据层而言，DWS层保留通用主题，ADS层保留应用组织方式。

数仓口径设计还需要处理指标命名一致性。不同页面若分别使用trip_num、count、total_count等字段表示出行量，后续维护时容易混乱。因此系统在DWS层统一使用trip_count表示出行次数，使用flow_count表示OD流量，使用avg_speed_kmh表示平均速度，使用congestion_index表示拥堵指数。统一字段命名能够减少接口转换和前端适配成本。

分层表之间还需要建立清晰的数据依赖关系。ODS表只依赖原始文件，DWD表依赖ODS表和维表，DWS表依赖DWD表，ADS结果依赖一个或多个DWS表。论文中说明依赖关系，可以帮助读者理解某个页面指标不是凭空产生，而是经过多级处理得到。若后续某个图表结果异常，也可以沿着依赖关系逐层排查。

在数据仓库实现中，时间维度可以进一步拆分为小时、日期、星期、月份和节

假日标识。当前系统主要使用小时和日期类型满足毕业设计需求，但保留 weekday、is_weekend 等字段后，后续可以扩展工作日与周末对比、月度趋势和节假日异常分析。字段设计时预留扩展空间，比后期反复改表更稳妥。

表 4-9 DWS 主题指标口径说明

主题表	核心指标	指标含义
dws_hourly_trip_count	trip_count	某日期、某小时、某交通方式下的出行次数
dws_hourly_trip_count	avg_duration_min	对应时间粒度下的平均出行时长
dws_od_flow	flow_count	起点到终点的出行记录数量
dws_hotspot_zone	hot_score	区域热度分值，由出行次数和空间权重综合得到
dws_road_congestion_rank	avg_speed_kmh	道路在统计时间范围内的平均速度
dws_road_congestion_rank	congestion_index	道路拥堵程度指标，数值越高表示越拥堵
dws_transfer_station	transfer_count	地铁站周边单车或短途接驳记录数量
dws_transfer_station	radius_m	站点接驳分析所使用的空间半径

4.4 数据库概念结构设计

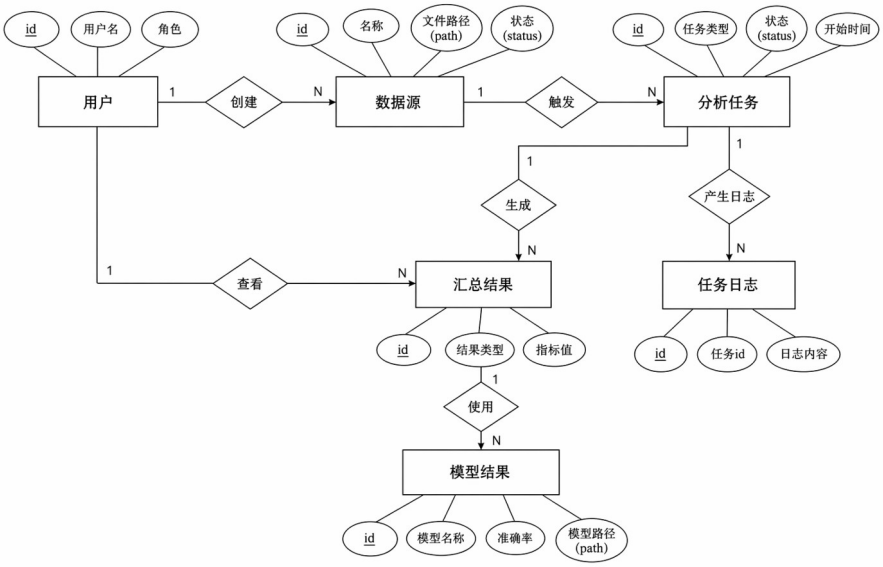

图 4-4 系统数据库 E-R 图

系统数据库主要保存业务元数据，而不是保存全部交通明细。交通明细数据体量较大，更适合放在HDFS、Hive或CSV结果文件中；MySQL主要保存用户、数据源、分析任务、任务日志、结果摘要和模型结果等信息。如图4-4所示，用户可以创建数据源和分析任务，分析任务产生日志和DWS结果，模型结果基于DWS特征输出预测指标。

用户表用于保存登录账号、角色和基础信息，是权限控制的基础。数据源表用于记录文件名、文件路径、数据类型、记录数和状态，便于前端展示数据来源。分析任务表记录任务类型、运行状态、开始时间、结束时间和参数，任务日志表保存任务执行过程。结果表和模型结果表则保存可视化页面或预测页面需要引用的摘要信息。

数据库概念结构中的实体关系体现了系统的业务流程。用户创建数据源，数据源被分析任务引用，分析任务产生任务日志和结果摘要，模型结果基于分析结果生成。这个流程与系统实际使用顺序一致，因此E-R图不仅是数据库设计图，也可以作为业务流程说明。

用户与分析任务之间是一对多关系，一个用户可以创建多个任务；数据源与分

析任务之间可以理解为多对多关系，一个任务可能读取多个数据源，一个数据源也可能被多个任务复用。为简化实现，系统可以在任务参数中保存数据源 ID 列表，而不单独建立中间表。

模型结果与分析任务之间通常是一对一或一对多关系。一次模型训练任务可能输出一个模型文件和一组指标，也可能在不同参数下输出多个模型结果。论文设计中将模型结果独立成表，可以方便前端展示历史模型和指标对比。

4.5 主要数据表设计

为避免数据库表只以清单形式堆叠，本文按照业务含义分别说明主要表的作用。用户表承担系统访问控制功能，字段不宜过多，重点是用户名、密码摘要、角色和登录时间等；数据源表承担文件登记功能，重点是文件路径、来源类型、状态和记录数；分析任务表承担后台任务管理功能，重点是任务类型、参数、状态、日志和结果路径。

表 4-2 用户表 user

字段名	类型	说明
id	INT	主键，自增
username	VARCHAR(64)	登录用户名，唯一
password_hash	VARCHAR(255)	密码摘要，不保存明文密码
role	VARCHAR(32)	用户角色，admin 或 analyst
email	VARCHAR(128)	用户邮箱
last_login_at	DATETIME	最后登录时间
created_at	DATETIME	创建时间

用户表的设计重点在于权限边界。管理员可以访问数据源维护、任务调度和用户管理等页面，普通分析用户则主要访问可视化和预测页面。通过 role 字段可以在后端接口中进行权限判断，也可以在前端导航中控制菜单展示。

表 4-3 数据源表 data_source

字段名	类型	说明
id	INT	主键，自增
name	VARCHAR(128)	数据源名称
source_type	VARCHAR(32)	generated、sample 或 uploaded
file_path	VARCHAR(255)	本地或 HDFS 文件路径
record_count	INT	记录数
status	VARCHAR(32)	ready、importing、failed 等状态
description	TEXT	数据源说明
created_at	DATETIME	创建时间

数据源表用于把文件和业务含义关联起来。系统中存在出租车、地铁、单车、道路、天气、站点和道路维表等多种文件，若只依赖文件目录，前端难以展示状态和备注信息。数据源表提供统一登记入口，后续 Spark 任务可以根据数据源记录读取输入路径。

表 4-4 分析任务表 analysis_task

字段名	类型	说明
id	INT	主键，自增
task_type	VARCHAR(64)	etl、hotspot、od、congestion、model_train 等
status	VARCHAR(32)	pending、running、success、failed
params	JSON	任务运行参数
result_path	VARCHAR(255)	结果文件或 Hive 表路径
start_time	DATETIME	任务开始时间
end_time	DATETIME	任务结束时间
created_by	INT	创建用户 ID

分析任务表体现系统后台计算过程。用户在页面发起任务后，后端记录任务状态并调用 Spark 或本地 fallback 脚本。任务运行过程中产生的日志写入 task_log 表，任务结束后将结果路径写回 analysis_task 表。这样前端可以展示任务进度、错误信息和结果入口。

任务日志表虽然字段简单，但对系统可维护性非常重要。Spark 任务可能因为输入路径错误、字段缺失、Hive 连接失败或权限问题而失败，如果没有日志表，前端只能看到失败状态，无法定位问题。通过记录 task_id、日志级别、日志内容和时间戳，管理员可以在页面上追踪后台执行过程。

表 4-5 任务日志表 task_log

字段名	类型	说明
id	INT	主键，自增
task_id	INT	所属分析任务 ID
log_level	VARCHAR(16)	INFO、WARN 或 ERROR
message	TEXT	任务执行日志内容
created_at	DATETIME	日志产生时间

模型结果表用于保存训练任务输出。该表不保存完整模型文件，而是保存模型名称、指标摘要、特征重要性、模型文件路径和创建时间。这样前端模型页面可以快速展示训练结果，预测接口则根据 artifact_path 加载模型文件。

表 4-6 模型结果表 model_result

字段名	类型	说明
id	INT	主键，自增
model_name	VARCHAR(64)	模型名称，如 random_forest
metric_json	JSON	准确率、RMSE、MAE 等指标

feature_importance	JSON	特征重要性列表
artifact_path	VARCHAR(255)	模型文件保存路径
created_at	DATETIME	创建时间

数据表设计要避免把所有信息都塞进一个大表。用户、数据源、任务、日志和模型结果分别建表，能够降低字段耦合。若把任务日志直接放在任务表中，日志内容会越来越长；若把模型指标直接放在任务表中，后续新增模型字段会使表结构混乱。分表设计更利于维护。

字段类型选择也需要与业务含义匹配。状态字段使用 VARCHAR 便于保存 pending、running、success 和 failed 等状态；参数字段使用 JSON 便于保存不同任务的差异化参数；时间字段使用 DATETIME 便于计算运行耗时；路径字段使用 VARCHAR 保存本地或 HDFS 路径。字段类型不是随意选择，而是服务于系统扩展。

数据库表与 Hive 表之间需要区分。MySQL 表保存系统管理信息和结果摘要，Hive 表保存数据分析明细和聚合结果。若把 5 万条甚至更多交通明细全部放入 MySQL，既不符合大数据系统设计，也会削弱 Hadoop 和 Spark 在论文中的作用。本文通过两类表的分工体现系统层次。

4.6 接口设计

系统接口采用 REST 风格，统一基础路径为/api/v1/。认证接口负责登录和用户

信息，数据源接口负责列表、上传和预览，任务接口负责创建和查询，数据可视化接口负责返回图表所需 JSON。所有接口使用统一响应结构，便于前端封装请求工具。

表 4-7 主要接口设计

模块	接口	功能说明
认证	POST /api/v1/auth/login	用户登录，返回访问令牌和用户信息
数据源	GET /api/v1/data-sources	分页查询数据源列表
数据源	GET /api/v1/data-sources/{id}/preview	预览数据源字段和样例记录
任务	POST /api/v1/analysis/tasks	创建 Spark 分析任务
可视化	GET /api/v1/viz/dashboard	获取综合大屏 KPI 和图表数据
可视化	GET /api/v1/viz/od-flow	获取 OD 流向统计结果
模型	POST /api/v1/model/predict	提交特征并返回拥堵预测结果
系统	GET /api/v1/system/health	检查依赖状态和服务健康

代码 4-1 接口统一返回结构示例

```
{
  "code": 0,
  "message": "ok",
  "data": {
    "kpis": {
      "total_trips": 45000,
      "road_records": 5000
    }
  }
}
```

接口返回结构中 code 用于标识业务状态，message 用于提示信息，data 用于承载实际数据。前端在处理图表数据时只依赖 data 字段内部的业务结构，错误状态则统一通过 code 和 message 处理。该设计可以减少页面中重复的异常处理代码。

接口设计需要稳定命名规范。数据源相关接口统一使用 data-sources，分析任务相关接口统一使用 analysis/tasks，可视化相关接口统一使用 viz 前缀，模型相关接口

统一使用 model 前缀。统一命名有利于前端封装请求，也便于论文表格中展示。

接口参数需要尽量业务化。例如 OD 接口使用 origin、destination、transport_type 和 limit，而不是要求前端传入底层表名；拥堵接口使用 road_name、date 和 limit，而不是暴露 Spark 输出文件路径。后端对底层数据路径进行封装，可以减少前端对数据存储细节的依赖。

接口异常处理也应统一。文件不存在、任务未完成、参数错误和权限不足都应返回约定格式，而不是直接返回框架默认错误页面。统一异常结构可以让前端以同一种方式提示用户，也能在论文中说明系统具有基本健壮性。

表 4-8 可视化接口参数说明

接口	主要参数	返回内容
GET /api/v1/viz/dashboard	date、area	总出行量、峰值小时、热点区域、拥堵道路
GET /api/v1/viz/hourly-trend	date、transport_type	按小时统计的出行量和平均时长
GET /api/v1/viz/hotspots	date、transport_type、limit	热点区域名称、坐标和热度权重
GET /api/v1/viz/od-flow	date、transport_type、limit	起点、终点、流量和交通方式
GET /api/v1/viz/congestion	date、road_name、limit	道路名称、平均速度和拥堵指数
GET /api/v1/viz/transfer	station_name、radius	站点周边接驳强度和短途出行量

5 系统详细设计与实现

详细设计与实现部分围绕系统核心模块展开，包括数据生成与清洗、Hive建表与数仓分层、Spark ETL任务、后端接口、前端可视化和模型预测。与只展示页面截图的系统不同，本系统的重点是把数据处理链路和页面展示结合起来，因此需要说明每个模块如何承接上一层输出，并为下一层提供输入。

5.1 数据生成与预处理实现

系统使用的数据主要位于data/generated目录，主业务数据包括出租车行程、地铁刷卡OD、共享单车订单和道路拥堵记录。数据生成脚本按照天津区域、站点、道路、天气和出行时间规律构造样本，并保留is_synthetic、source_basis、generation_version和quality_flag等字段，用于说明数据来源和质量标记。

数据预处理首先进行字段完整性检查和类型转换。出租车数据需要检查上下车经纬度是否落在合理范围内，地铁数据需要检查进出站字段是否为空，单车数据需要检查起终点坐标和最近地铁站，拥堵数据需要检查拥堵指数和平均速度是否为正值。随后根据不同数据源的时间字段提取日期、小时、星期和日期类型。

表5-1 数据预处理规则

数据源	清洗规则	输出字段
出租车行程	过滤空trip_id，限定天津经纬度	record_id、origin_name、

	范围，转换上下车时间	destination_name、duration_min
地铁刷卡	过滤空 trip_id，统一进出站名称，转换进出站时间	record_id、origin_name、destination_name、start_time
共享单车	过滤空 order_id，统一起点区域和最近地铁站	record_id、origin_lng、origin_lat、distance_km
道路拥堵	过滤无效拥堵指数，保留道路、速度和天气字段	road_name、avg_speed_kmh、congestion_index

数据生成过程需要保证业务规律可解释。例如工作日早晚高峰的出行量应高于深夜，雨天道路平均速度应低于晴天，商圈、交通枢纽和地铁站点周边应具有更高热度。只有当这些规律在数据中体现出来，后续可视化页面才不会显得随机，也能在论文分析中说明系统输出具有合理性。

预处理阶段还需要记录异常处理策略。对于关键字段为空的记录，系统通常直接过滤；对于可为空的辅助字段，可以保留 NULL 并在后续聚合中处理；对于数值字段异常值，可以通过范围判断剔除。该策略能够保证 DWD 层数据质量，同时避免过度清洗导致样本量大幅下降。

数据生成脚本需要按照业务规则控制字段分布。例如工作日早高峰地铁和单车数据增加，夜间出租车比例提高，雨天道路拥堵指数提高，交通枢纽和商圈周边热点权重提高。这些规则不是为了伪造真实数据，而是为了让仿真数据具备可解释规律，使系统图表能够反映合理趋势。

预处理实现中，时间字段转换是最容易出错的环节。不同 CSV 可能使用不同格式，如 YYYY-MM-DD HH:mm:ss、时间戳或带日期的字符串。系统在清洗阶段统一转换为标准时间类型，并从中提取 dt、hour_of_day 和 weekday 字段。若转换失败，记录会被标记为异常并进入日志。

坐标字段检查需要结合天津范围。系统设置经纬度边界，过滤明显越界坐标，并对缺失坐标的记录进行处理。对于缺失起点或终点坐标但有站点名称的记录，可以通过站点维表补充；对于无法补充的记录，则不参与地图类图表计算，但仍可在非空间统计中保留。

数据预处理还包括单位统一。距离字段统一为 km，时长字段统一为分钟，速度字段统一为 km/h，降雨量统一为 mm。单位统一可以避免不同数据源之间计算口径不一致，也方便论文中解释指标含义。

5.2 Hive 分层表实现

Hive 表实现包括 ODS、DWD 和 DWS 三个主要层次。ODS 表采用外部表方式映射 CSV 文件，便于保留原始数据；DWD 表采用 ORC 存储并按 dt 分区，便于提高查询效率；DWS 表则以主题聚合结果为主，面向前端和模型模块提供稳定输入。

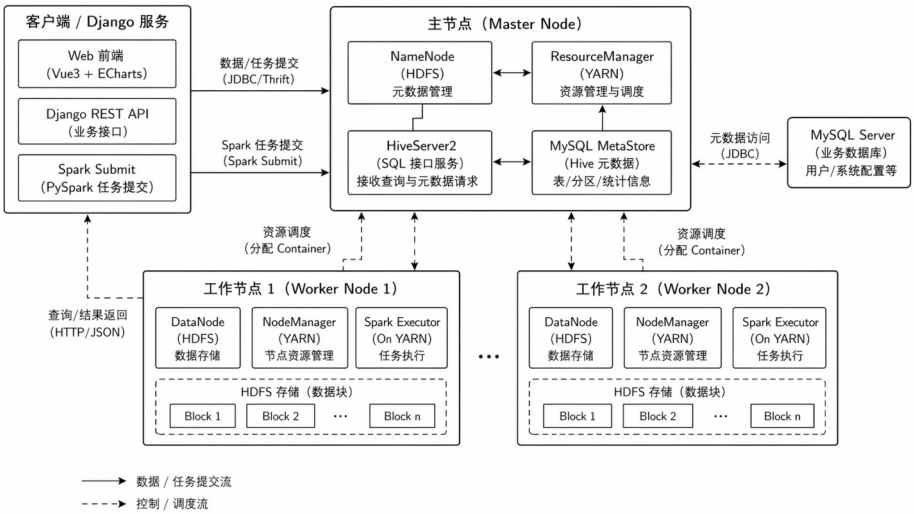

图 5-1 Hadoop 与 Spark 部署结构图

如图 5-1 所示，系统在平台环境中可采用主从式大数据部署结构。Master 节点负责 NameNode、ResourceManager、HiveServer2 和元数据库，Worker 节点负责 DataNode、NodeManager 和 Spark Executor。Django 服务通过 Hive JDBC 或 Spark Submit 与大数据组件交互，既能查询离线结果，也能触发批处理任务。

代码 5-1 DWD 出行明细表核心字段

```
CREATE TABLE IF NOT EXISTS dwd_trip_detail (
   record_id       STRING,
   transport_type   STRING COMMENT 'taxi/subway/bike',
   origin_name     STRING,
   destination_name STRING,
   origin_lng      DOUBLE,
   origin_lat      DOUBLE,
   dest_lng        DOUBLE,
   dest_lat        DOUBLE,
   start_time      TIMESTAMP,
   end_time        TIMESTAMP,
   duration_min     DOUBLE,
   distance_km      DOUBLE,
   hour_of_day      INT,
   day_type        STRING,
   weather_level    STRING
) PARTITIONED BY (dt STRING)
```

```
STORED AS ORC;
```

DWD 出行明细表的设计目标是统一出租车、地铁和单车三类出行记录。不同源表在字段命名上差异较大，但清洗后都能够转化为起点、终点、开始时间、结束时间、时长、距离、天气和日期类型等字段。通过 transport_type 字段保留交通方式，系统既能对全部出行进行统计，也能按交通方式进行对比分析。

Hive 建表脚本需要与数据生成字段保持一致。如果 CSV 字段变化而 Hive 表未同步更新，后续 Spark 读取会出现字段为空或类型不匹配问题。因此项目中将建表 SQL、数据生成脚本和 Spark 清洗逻辑放在同一仓库中管理，便于同时修改和追踪。

分区字段 dt 来自记录日期，而不是文件导入日期。这样设计能够保证同一天的交通记录进入同一分区，即使文件实际导入时间不同，也不会影响按日期查询。对于交通分析而言，业务日期比导入日期更有意义。

DWD 表中的 source_dataset 字段用于追踪记录来源。由于统一出行明细表合并了出租车、地铁和单车记录，后续如果某个统计结果异常，需要能够回溯具体来源。source_dataset 和 record_id 组合起来，可以定位到原始数据中的记录。

Hive 层还需要考虑字段注释。虽然字段注释不会影响程序运行，但在论文和后

续维护中非常有用。通过 COMMENT 说明字段含义，可以让阅读者理解每个字段的业务作用，也能体现表结构设计的规范性。

5.3 Spark ETL 任务实现

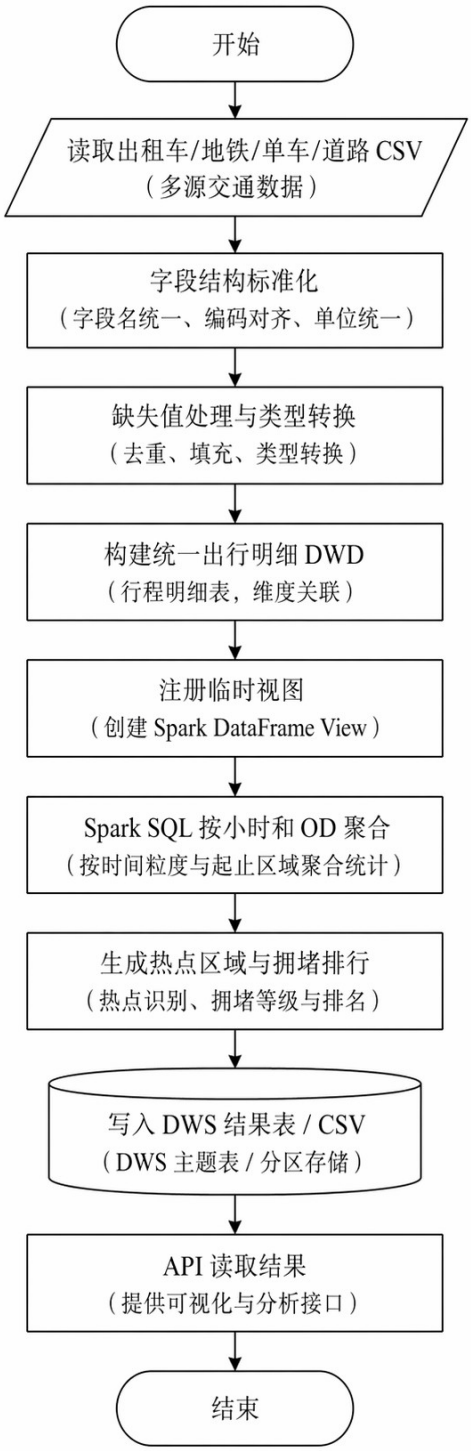

图 5-2 Spark ETL 任务流程图

Spark ETL 任务是系统数据计算的核心。如图 5-2 所示，任务首先创建

SparkSession 并开启 Hive 支持，然后读取出租车、地铁、单车和道路拥堵 CSV 文件；随后调用 clean_trips 函数将三类出行记录统一为 DWD 明细；最后分别构建小时出行量、OD 流向、道路拥堵排名和热点区域等 DWS 结果。

代码 5-2 Spark 统一出行明细核心逻辑

```
def clean_trips(taxi, subway, bike):
    taxi_df = taxi.select(
        col("trip_id").alias("record_id"),
        lit("taxi").alias("transport_type"),
        col("pickup_zone_name").alias("origin_name"),
        col("dropoff_zone_name").alias("destination_name"),
        col("pickup_time").cast("timestamp").alias("start_time"),
        col("duration_min").cast(DoubleType())
    )
    subway_df = subway.select(
        col("trip_id").alias("record_id"),
        lit("subway").alias("transport_type"),
        col("entry_station_name").alias("origin_name"),
        col("exit_station_name").alias("destination_name"),
        col("entry_time").cast("timestamp").alias("start_time"),
        col("duration_min").cast(DoubleType())
    )
    return taxi_df.unionByName(subway_df).unionByName(bike_df)
```

在 DWS 层构建中，小时趋势表按照 dt、hour_of_day 和 transport_type 分组，统计 trip_count、avg_duration_min 和 avg_distance_km；OD 流向表按照 origin_name、destination_name 和 transport_type 分组，统计 flow_count；道路拥堵表按照 road_name 分组，计算平均拥堵指数、平均速度和最大拥堵指数；热点区域表按照 origin_name 聚合，计算出行次数和平均坐标。

Spark任务实现中还设置了两种输出方式。当传入output_dir参数时，任务将DWS结果写成本地CSV文件，便于本地开发和前端演示；当传入output_db参数时，任务将结果写入Hive数据库，便于大数据平台环境下统一查询。这种双模式输出使系统能够兼顾论文设计完整性和实际运行便利性。

在性能优化方面，Spark任务启用了自适应执行，并使用Kryo序列化提高运行效率。虽然当前数据规模为5万条业务明细，单机即可处理，但保留这些配置可以体现系统面向更大数据规模扩展的考虑。后续数据量增大时，可以通过增加分区、缓存中间DataFrame和优化Hive表存储格式继续提升性能。

ETL任务输出完成后，后端不需要重新计算OD和热点，只需要读取DWS结果。这样的实现能够减少请求延迟，也能避免多个用户同时访问页面时重复触发离线计算。对于交通数据系统而言，离线批处理和在线查询的边界必须清楚，否则系统在演示时容易出现卡顿。

Spark ETL代码采用分函数组织。read_sources负责读取输入文件，clean_trips负责统一三类出行记录，build_hourly负责生成小时趋势，build_od_flow负责生成OD流向，build_hotspots负责生成热点区域，build_congestion负责生成拥堵排名。

每个函数只处理一类任务，便于测试和论文说明。

在清洗阶段，Spark使用withColumn补充派生字段。例如通过hour(start_time)生成hour_of_day，通过when表达式生成day_type，通过join天气表补充weather_level。派生字段不直接写死在前端，而是在DWD或DWS层形成稳定结果，使后续分析保持统一。

在聚合阶段，Spark需要对异常值进行保护。例如计算平均速度时要过滤小于0的速度，计算平均时长时要过滤极端长时间记录，计算OD流量时要过滤起点或终点为空的记录。异常过滤规则应在论文中说明，否则读者可能质疑统计结果来源。

Spark任务输出还需要保证覆盖写入和增量写入的边界。演示环境中可以使用overwrite覆盖旧结果，保证每次运行得到最新数据；生产环境中则更适合按dt分区追加，避免覆盖历史数据。本文主要采用覆盖写入用于演示，同时在设计中说明后续可扩展为分区追加。

Spark ETL的调试过程通常不是一次完成的。首先需要确认每个源文件能被正确读取，其次需要确认字段类型转换没有造成大量空值，再次需要确认union后的DWD宽表字段数量和记录数符合预期，最后才进入DWS聚合阶段。若跳过中间检

查，最终图表异常时很难判断问题发生在读取、清洗还是聚合环节。

任务运行日志应尽量记录关键统计信息。例如读取出租车记录数、清洗后记录数、过滤异常记录数、输出 DWD 记录数、输出 DWS 表数量和任务耗时等。这些信息不仅便于调试，也能作为论文中“系统确实执行了数据处理流程”的证据。相比只写代码片段，日志统计更能体现工程实现过程。

Spark 输出结果还要注意字段顺序。前端和后端读取 CSV 结果时通常按字段名解析，但人工检查和论文截图往往依赖字段顺序。系统在输出 DWS 结果时将核心字段放在前面，例如日期、小时、交通方式、指标值，再放辅助字段。这样既方便程序读取，也方便论文说明。

对于热点聚合，系统不直接把所有经纬度点传给前端，而是先按区域或网格聚合。这样可以减少前端地图渲染压力，也能避免点位过多导致热力图视觉混乱。聚合后的 hot_score 能够表达区域强度，平均经纬度则用于地图定位。

对于 OD 流向，系统默认按 flow_count 排序并取 TopN。完整 OD 矩阵可能包含大量组合，如果全部绘制，图表线条会严重重叠。TopN 策略能够突出主要流向，符合可视化表达原则。论文中说明这一点，可以避免评阅教师认为系统遗漏了其他

OD 记录。

表 5-2 Spark 任务日志关键检查项

检查项	记录内容	用途
输入文件检查	输入路径、文件存在状态、读取记录数	判断数据源是否可用
字段检查	缺失字段、字段类型转换失败数量	定位清洗异常
DWD 输出检查	统一明细记录数、交通方式分布	验证多源合并结果
DWS 输出检查	各主题表记录数、Top 指标示例	验证聚合结果可用于页面
耗时检查	任务开始时间、结束时间、总耗时	评估离线计算效率
异常日志	错误类型、错误位置、异常消息	辅助管理员排查任务失败原因

5.4 后端接口实现

后端使用 Django REST Framework 实现统一 API，项目中 TrafficAnalyticsService 承担可视化数据聚合职责。该服务可以读取出租车、地铁、单车、道路、区域、站点和道路维表，并根据不同接口返回仪表盘、热力图、时间序列、OD 流向、聚类和拥堵排名等结果。

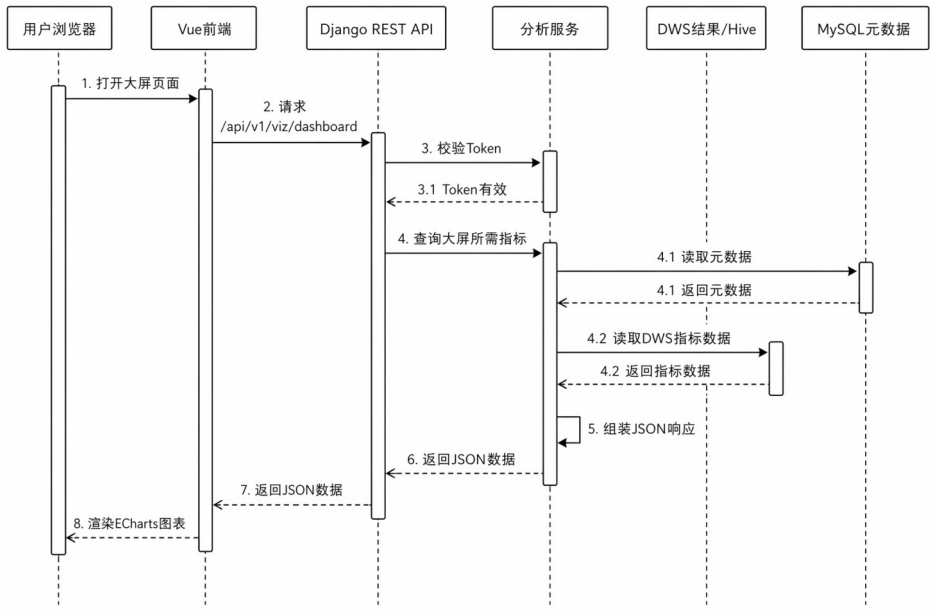

图 5-3 可视化接口调用时序图

如图 5-3 所示，用户打开可视化页面后，Vue 前端向 Django REST API 发送请求，后端先进行身份校验，再由分析服务读取 MySQL 元数据和 DWS 结果文件或 Hive 结果表，最后组装为 JSON 返回给前端。前端拿到数据后构建 ECharts 配置并完成页面渲染。

仪表盘接口会读取出租车、地铁、单车和道路四类数据，计算 total_trips、taxi_trips、subway_trips、bike_orders 和 road_records 等 KPI，同时返回小时趋势、热点区域、平均指标、天气分布和拥堵 Top5。前端综合大屏通过一次接口调用即可获得多块图表所需数据，降低了页面多接口并发请求的复杂度。

热力图接口通过_extract_points 函数提取不同数据源的坐标点，并根据数据类型

赋予不同权重。例如出租车上车点权重可高于普通点，地铁进出站点用于体现站点客流，共享单车起终点用于体现短途活动，道路拥堵点则以拥堵指数作为权重。最后通过聚合函数限制点数，保证地图展示性能。

后端服务在读取 CSV 或结果表时，还需要对空值和字段缺失进行保护。由于数据源可能处于未导入或任务未完成状态，接口应在数据为空时返回空数组和统计为 0 的指标，而不是直接抛出异常。这样前端页面可以展示空态提示，系统整体体验更稳定。

对于 OD 流向、热力点和排名数据，后端还需要限制返回数量。地图热力点如果一次返回过多，会导致浏览器渲染压力增大；OD 流向如果不做 Top 筛选，也会让图表难以阅读。因此服务层通过 head、limit 或 TopN 参数控制结果规模，在准确性和可读性之间取得平衡。

后端接口实现中，序列化器用于约束输入输出字段。数据源创建接口需要校验名称、类型和路径；任务创建接口需要校验任务类型和参数；预测接口需要校验道路、小时和天气等特征。通过序列化器校验，可以减少无效请求进入业务逻辑。

服务层读取结果文件时需要处理文件不存在情况。若用户尚未执行 ETL 任务，

DWS 结果文件可能不存在，此时接口返回空数据和提示信息，而不是抛出服务器错误。这样的容错逻辑对答辩演示非常重要，因为演示环境经常会出现路径或服务启动顺序问题。

后端还需要处理跨源数据的格式转换。Spark 输出的 CSV 字段可能使用下划线命名，前端图表需要的字段可能更适合使用 name、value、children 等结构。服务层负责把底层字段转换为图表友好结构，前端无需知道 DWS 表的全部字段。

日志接口用于展示后台任务执行过程。任务启动、读取文件、生成 DWD、生成 DWS、写入结果和任务结束等步骤都可以记录为 INFO 日志；字段缺失、空数据和异常值较多可以记录为 WARN；任务失败则记录 ERROR。日志信息越清晰，系统越容易维护。

5.5 前端可视化实现

前端采用 Vue3、TypeScript、Naive UI 和 ECharts 实现。页面整体使用 AppShell 布局，左侧或顶部导航包含概览、大屏、数据源、热点、趋势、OD、拥堵、接驳、数仓、模型、预测和用户等模块。系统主题色保持清晰克制，适合数据分析场景。

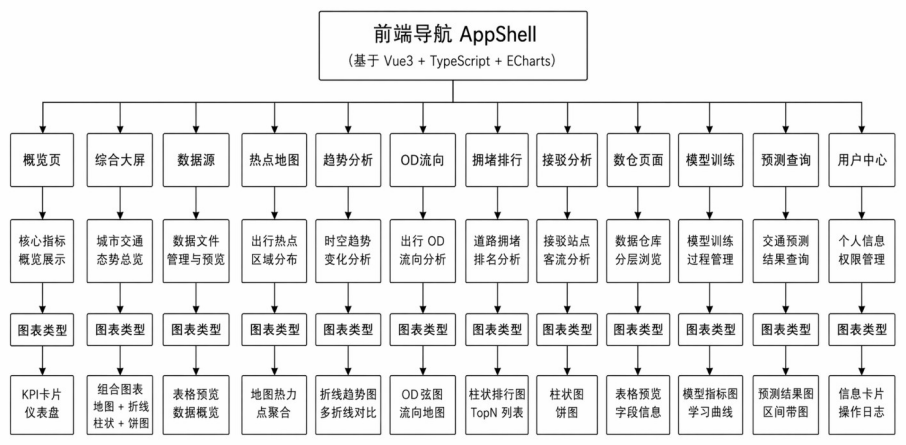

图 5-4 前端页面组织结构图

如图 5-4 所示，前端页面按照业务专题组织，而不是按照技术组件堆叠。概览和大屏用于总览展示，数据源页面用于查看文件和状态，热点、趋势、OD、拥堵和接驳页面分别对应 DWS 层的主题结果，数仓和模型页面用于说明后台处理过程和模型输出。

综合大屏是系统展示的核心页面，主要呈现 KPI、天津区域热力图、出行方式趋势、道路拥堵分布和 Top 排名。专题页面则根据分析对象拆分：热点页面关注区域热度，趋势页面关注小时变化，OD 页面关注起终点流向，拥堵页面关注道路排名，接驳页面关注地铁站周边单车联系，模型页面展示训练指标和预测结果。

前端实现时需要注意图表与接口结构的一致性。后端返回的 series、points、flows 和 rank 等字段应直接映射到 ECharts 配置中，避免在页面中写大量数据清洗代

码。对于空数据状态，页面应展示加载提示或空态组件，而不是让图表区域报错。

前端页面设计遵循“总览先行、专题下钻”的思路。用户进入系统后先看到综合大屏，对总体数据量、趋势和热点有基本认识；随后可根据关注点进入趋势、OD、拥堵、接驳和模型页面查看细节。这种组织方式比把所有图表堆在一个页面上更清晰，也更符合答辩讲解顺序。

图表交互上，系统应支持基础筛选和提示信息。例如趋势图可根据交通方式切换展示，OD 页面可查看 Top 流向，拥堵页面可查看道路名称、平均速度和拥堵指数，预测页面可根据用户输入更新结果。交互功能不宜过度复杂，但要能体现系统不是静态截图，而是可操作的应用。

前端综合大屏页面采用指标卡、趋势图、地图和排行榜组合展示。指标卡显示总出行量、道路记录数、热点区域数和模型准确率等核心指标；趋势图显示小时变化；地图显示空间热点；排行榜显示拥堵道路或 OD 流量。多种图表组合能够从不同角度展示系统结果。

趋势分析页面重点展示时间维度。用户可以观察一天中不同交通方式的出行峰谷，也可以对比工作日和非工作日。该页面的数据来源于 dws_hourly_trip_count，前

端只负责根据 transport_type 绘制不同折线，减少复杂计算。

OD 分析页面重点展示空间流向。系统将起点、终点和流量作为主要字段，前端可用表格、柱状图或流向图展示 Top 流向。由于 OD 组合可能很多，页面默认只展示 TopN，避免图表过密。

拥堵分析页面重点展示道路状态。页面可以展示平均速度、拥堵指数、拥堵等级和道路名称，并根据拥堵指数排序。用户通过该页面可以快速判断哪些道路在样本数据中表现出较高拥堵风险。

模型预测页面重点展示输入和输出。用户选择道路、时间、天气等特征后，系统返回预测结果和风险提示。页面还可以展示模型特征重要性，使用户理解哪些因素对预测结果影响较大。

前端页面的状态管理需要避免重复请求。综合大屏加载时会请求多个图表数据，如果每个组件各自独立请求，可能造成接口压力和状态不一致。系统可以在页面级统一请求 dashboard 接口，再将结果分发给各个图表组件。专题页面则根据用户筛选条件重新请求对应接口。

图表组件需要处理窗口变化。答辩时经常会在不同分辨率屏幕或投影设备上展

示页面，若图表不随容器变化重新渲染，可能出现图表溢出或空白。前端通过 resize 监听或 ECharts 实例 resize 方法保证图表适配容器。

页面路由设计也要符合用户使用习惯。数据源、任务和系统状态属于管理类页面，综合大屏、热点、趋势、OD 和拥堵属于分析类页面，模型训练和预测属于模型类页面。导航按这些类别组织，用户可以更快找到目标功能，也方便论文按模块截图展示。

前端还需要统一颜色和图例。出租车、地铁、单车和道路拥堵应使用固定颜色，否则不同图表中的同一交通方式颜色变化会造成理解困难。系统在图表配置中统一 transport_type 到颜色的映射，提高整体视觉一致性。

表 5-3 前端页面组件划分

页面	核心组件	数据来源
综合大屏	KPI 卡片、趋势图、热点地图、拥堵排行	/api/v1/viz/dashboard
数据源页面	数据源表格、上传对话框、字段预览	/api/v1/data-sources
任务页面	任务列表、日志抽屉、状态标签	/api/v1/analysis/tasks
趋势页面	折线图、交通方式筛选、时间筛选	/api/v1/viz/hourly-trend
OD 页面	Top 流向表、流向图、方式筛选	/api/v1/viz/od-flow
拥堵页面	道路排行榜、速度柱状图、等级标签	/api/v1/viz/congestion
模型页面	训练结果表、特征重要性、预测表单	/api/v1/model

5.6 模型训练与预测实现

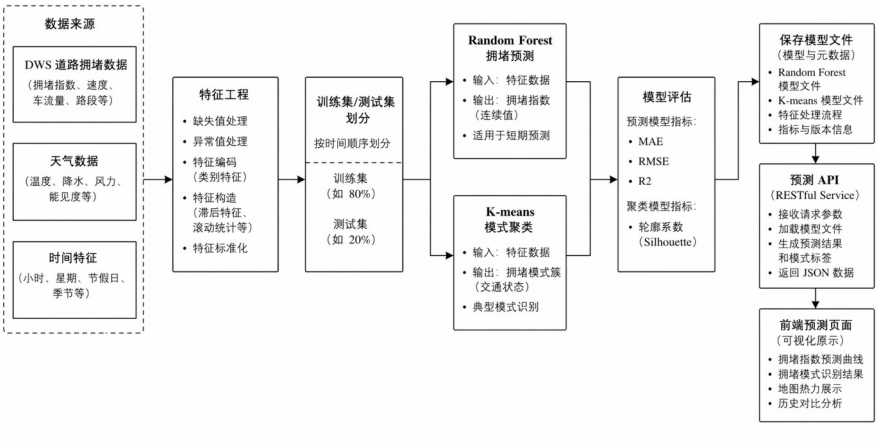

图 5-5 拥堵预测模型流程图

模型模块以道路拥堵数据为基础，训练入口位于 ml/train/train_congestion.py。脚本默认读取 data/generated/tianjin_road_congestion_5000.csv，通过 load_congestion_dataset 完成特征读取和预处理，再调用 train_and_select 训练并选择模型。训练结果保存到 ml/artifacts/congestion 目录，包含模型文件、评估指标和特征重要性。

拥堵预测特征主要包括时间、道路、天气和速度相关字段。时间特征用于反映早晚高峰，天气特征用于反映雨天和强降雨对通行速度的影响，道路等级和历史速度用于反映不同道路的基础通行能力。模型输出可用于预测拥堵等级或拥堵指数，前端预测页面再将结果以指标卡和图表方式呈现。

K-means 聚类主要用于热点区域或交通节点分群。系统可将经纬度、热度分值

和交通类型编码作为输入特征，对区域、站点和道路节点进行聚类。聚类结果不用于给出强因果解释，而是作为辅助分析手段，用于观察中心城区、交通枢纽和外围区域之间的空间结构差异。

模型训练实现首先构造特征集。道路拥堵记录与天气数据按时间字段关联，生成 hour_of_day、weather_level、precipitation_mm、avg_speed_kmh 和历史拥堵指数等特征。为了便于模型处理，类别字段需要进行编码，数值字段需要进行缺失值填补。

模型评估不只关注单一准确率。若预测目标是拥堵等级，可以使用 accuracy、precision、recall 和 F1；若预测目标是拥堵指数，可以使用 MAE、RMSE 等回归指标。论文中以系统展示为主，因此模型指标作为功能证明，而不夸大算法性能。

训练结果需要保存模型文件和指标摘要。模型文件用于后续预测接口加载，指标摘要用于前端页面展示，特征重要性用于解释模型。若只在训练脚本中打印指标，前端页面无法复用结果，也不利于论文展示。

预测接口应避免直接暴露模型内部细节。用户提交业务特征后，后端完成编码、模型加载和预测，再返回拥堵等级、概率或指数。前端只展示结果和提示，不参与

模型计算。这样能够保持前后端职责清晰。

6 系统测试与运行结果分析

系统测试的目标是验证数据链路、后端接口、前端页面和模型模块能否形成可运行闭环。由于本系统同时包含大数据处理和 Web 展示，测试不能只关注页面是否打开，还需要检查数据行数、字段口径、Spark 输出结果、接口返回结构和图表展示效果。本文结合项目文档、数据文件和实际运行视频截图进行测试说明。

6.1 测试环境

表 6-1 系统测试环境

项目	配置
操作系统	macOS / Linux 本地开发环境
Python	3.10 及以上
Node.js	18 及以上
后端框架	Django REST Framework
前端框架	Vue3、TypeScript、Naive UI、ECharts
大数据组件	Hadoop 3.3+、Hive 3.1+、Spark 3.4+
数据库	MySQL 8.0 或 SQLite 开发库
浏览器	Chrome、Edge 或 Safari

测试环境既支持完整大数据组件运行，也支持本地 fallback 模式。在毕业设计演示中，本地模式可以直接读取 data/generated 和 data/dws 中的文件，保证页面展示稳定；在平台环境中，Spark 任务可以切换到 HDFS 和 Hive 路径，体现真实大数据处理流程。

测试环境需要记录软硬件配置和依赖版本。大数据系统涉及 Python、Spark、

Hive、Django、Node.js 和浏览器环境，如果不说明环境，后续复现时可能因为版本差异出现问题。论文中列出主要环境，可以让评阅教师理解系统运行条件。

由于完整 Hadoop 集群部署较复杂，本文测试采用本地演示和大数据设计相结合的方式。本地环境负责验证前端页面、后端接口和结果读取，大数据设计负责说明 HDFS、Hive 和 Spark 在完整系统中的位置。该方式符合毕业设计演示的实际条件，也降低了环境不稳定风险。

测试数据采用固定生成版本，保证多次运行结果基本一致。若每次生成数据都完全随机，截图、表格和论文文字可能不一致。通过记录 generation_version 和随机种子，可以让系统结果具有可复现性。

6.2 数据质量测试

数据质量测试主要检查行数、坐标范围、字段完整性、时间分布和真实性报告。项目中 tests/test_tianjin_synthetic_data.py 覆盖了数据行数、坐标边界、高峰分布、来源字段、天气表、地铁拓扑、GPS 轨迹点、数值范围和真实性报告输出等内容。

表 6-2 数据质量测试用例

用例编号	检查项	预期结果	结果
DQ-01	主业务数据行数	出租车、地铁、单车各 15000，道路 5000	通过
DQ-02	坐标范围	经纬度落在天津合理范围内	通过
DQ-03	峰谷特征	7/8/17/18 点高于深夜	通过
DQ-04	来源字段	is_synthetic、source_basis 等	通过

		字段存在	
DQ-05	天气数据	31天24小时完整覆盖	通过
DQ-06	真实性报告	realism_report.json可生成	通过

测试结果表明，数据能够满足系统演示和算法验证需要。需要强调的是，数据质量测试证明的是仿真数据的结构合理性和分布可用性，并不代表该数据是天津官方真实运营记录。因此论文结论应围绕系统设计、功能实现和分析方法展开，不将具体数值扩展为真实交通治理结论。

数据质量测试首先检查记录数是否符合预期。出租车、地铁、单车和道路拥堵数据的记录数需要与数据源表中的record_count一致，若差异过大，说明导入或清洗过程存在问题。记录数检查是最基础但也最重要的测试项。

坐标范围测试用于保证地图展示合理。系统检查经纬度是否落在天津合理范围内，并统计越界记录数量。若越界点过多，热力图会出现偏移，影响可视化可信度。通过坐标范围测试，可以证明地图数据经过基础校验。

峰谷特征测试用于验证仿真数据规律。工作日早晚高峰出行量应高于深夜，雨天拥堵指数应高于晴天，站点和商圈周边热点应高于普通区域。该测试不追求与真实城市完全一致，而是验证数据是否具备交通分析所需的基本规律。

字段完整性测试用于检查关键字段缺失情况。record_id、transport_type、

start_time、origin_name 等字段是后续分析基础，如果缺失比例过高，DWS 结果会失真。系统通过统计缺失率判断数据是否可进入下一步计算。

数据质量测试还需要检查跨表关联结果。天气表按小时与出行明细关联，如果时间字段格式不一致，weather_level 字段会大量为空；地铁站点维表与单车接驳分析关联，如果站点名称不一致，接驳结果会偏少。因此测试中不仅检查单表记录，也检查关联后字段是否正常。

测试过程中还应比较清洗前后记录数变化。适度过滤异常记录是正常的，但如果过滤比例过高，可能说明清洗规则过严或数据生成字段不匹配。论文中可以说明过滤规则和过滤比例，使读者理解数据质量控制不是随意删除记录。

对于仿真数据，质量测试还要关注规律是否成立。例如雨天平均拥堵指数是否高于晴天，早晚高峰出行量是否高于凌晨，中心区域热点分值是否高于普通区域。如果这些规律不成立，系统图表即使能显示，也缺少业务解释力。

表 6-2 数据质量测试补充项

测试项	测试方法	通过标准
记录数一致性	比较源文件记录数、ODS 记录数和 DWD 记录数	差异在清洗规则可解释范围内
时间字段有效性	统计 start_time、end_time 转换失败数量	关键时间字段缺失率低于设定阈值
坐标范围有效性	检查经纬度是否位于天津合理范围	越界记录被过滤或标记
天气关联有效性	统计 DWD 中 weather_level 为空比例	关联缺失比例较低且原因可解释

峰谷规律检查	比较高峰、平峰和凌晨出行量	高峰出行量明显高于凌晨
拥堵规律检查	比较雨天和晴天平均拥堵指数	雨天拥堵指数不低于晴天

6.3 功能测试

表 6-3 系统功能测试用例

用例编号	功能	测试内容	结果
FT-01	综合大屏	查看 KPI、地图、趋势和排名模块	通过
FT-02	数据源管理	查看数据源列表和状态信息	通过
FT-03	趋势分析	展示小时趋势和交通方式对比	通过
FT-04	OD 分析	展示起终点流向和排名结果	通过
FT-05	拥堵分析	展示道路拥堵趋势、分布和 Top 列表	通过
FT-06	模型预测	输入参数后展示预测结果和模型说明	通过
FT-07	模型详情	查看模型指标、特征重要性和结果分析	通过

功能测试以实际运行视频为依据。从视频帧可以看到系统页面能够正常加载，综合大屏、数据管理、趋势分析、OD 分析、拥堵分析、模型预测和模型详情等页面均有对应展示。页面布局整体稳定，图表和指标卡能够支撑毕业设计演示。

功能测试还需要关注页面之间的跳转是否顺畅。根据实际运行视频，系统导航能够在综合大屏、数据源、趋势、OD、拥堵、模型和预测等页面之间切换，页面主体内容能够正常呈现。对于毕业设计展示而言，页面切换稳定性与图表本身同样重要，因为答辩时往往需要连续演示多个模块。

接口测试的重点是返回结构稳定。即使某些环境下后端读取的是本地 DWS 文

件，而另一些环境下读取的是Hive结果表，接口响应字段也应保持一致。前端只依赖统一结构，可以减少部署环境变化带来的适配问题。

功能测试采用“模块功能+输入条件+预期结果”的方式组织。数据源模块测试文件登记和预览是否正常，任务模块测试任务创建和状态更新是否正常，可视化模块测试图表数据是否能够加载，模型模块测试预测接口是否能够返回结果。

页面测试不仅关注是否能打开，还关注页面内容是否与接口数据对应。例如综合大屏中的总出行量应与接口返回KPI一致，趋势图中的小时数据应与DWS结果一致，拥堵排行榜的排序应按拥堵指数从高到低排列。只有页面和数据一致，运行截图才具有证明作用。

接口测试重点检查参数边界。limit为空、date为空、transport_type不存在、数据源ID不存在、任务ID不存在等情况都需要返回合理提示。虽然毕业设计不一定覆盖所有边界，但论文中列出典型测试用例可以体现系统鲁棒性。

任务测试重点检查状态流转。任务创建后状态为pending，开始执行后变为running，完成后变为success，失败后变为failed并产生错误日志。状态流转正确，前端才能展示任务进度，管理员也能定位问题。

表6-3 功能测试用例补充表

测试模块	测试内容	预期结果
数据源管理	登记出租车 CSV 并预览字段	返回字段列表和样例记录，状态为 ready
数据源管理	上传字段缺失的数据文件	返回校验失败提示，日志记录缺失字段
任务调度	创建 ETL 分析任务	任务状态由 pending 变为 running 并最终 success
任务调度	输入不存在的数据源 ID	接口返回参数错误，不创建无效任务
综合大屏	请求 dashboard 接口	返回 KPI、趋势、热点和拥堵排行数据
OD 分析	设置 limit=10 查询 Top 流向	返回不超过 10 条 OD 记录且按流量排序
模型预测	提交道路、天气和时间特征	返回预测等级、模型名称和提示信息
系统健康	访问 health 接口	返回数据库、结果目录和服务状态

6.4 运行效果展示

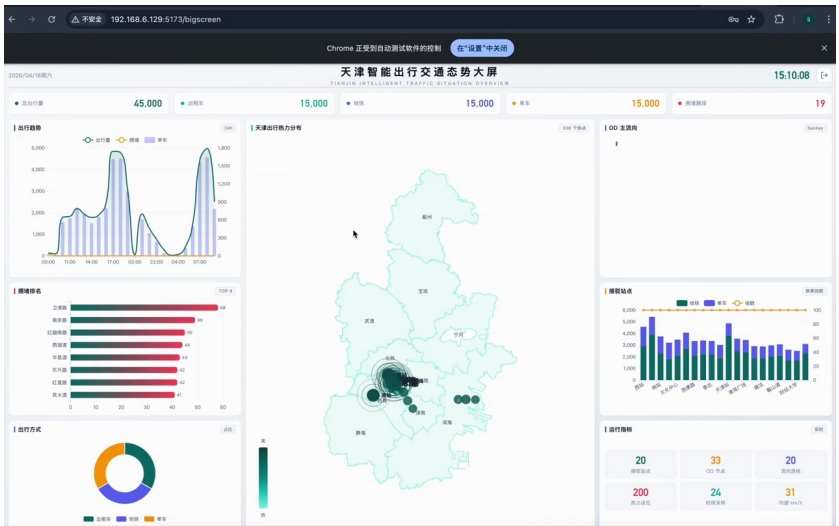

图 6-1 系统综合大屏运行效果

图 6-1 展示了系统综合大屏页面。该页面将核心 KPI、地图热力、出行方式趋势、道路拥堵和排名信息集中呈现，适合在答辩时快速说明系统整体功能。大屏页面的价值在于把多源交通数据压缩为可浏览的摘要结果，使用户能够先获得整体判断，

再进入专题页面查看细节。

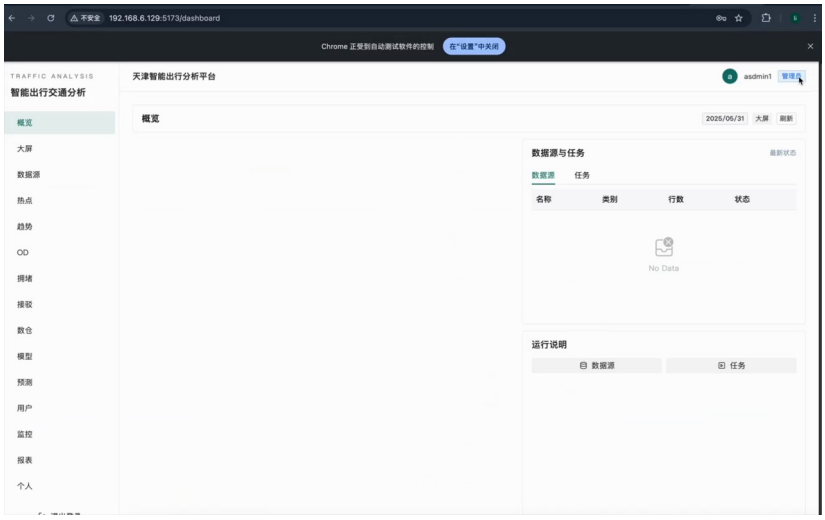

图 6-2 数据源管理页面运行效果

图 6-2 展示了数据源管理页面。该页面用于展示数据文件、导入状态和预览入口，是系统从静态图表走向完整数据管理流程的重要部分。

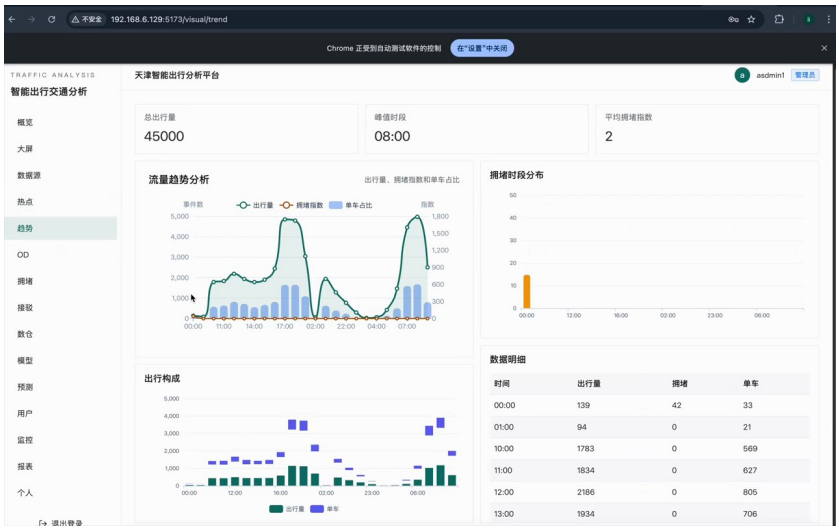

图 6-3 交通趋势与总览页面运行效果

图 6-3 展示了趋势和总览类页面，主要用于观察不同时间段的交通变化。通过折线图和统计表，可以对比出租车、地铁和共享单车在一天内的出行强度变化，并

结合道路拥堵指数观察早晚高峰特征。

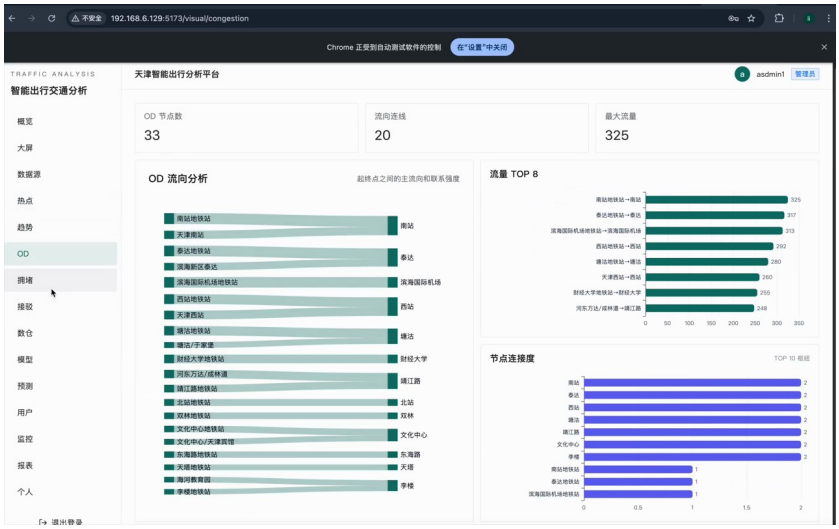

图 6-4 OD 流向与排行页面运行效果

图 6-4 展示了 OD 流向分析页面。OD 分析能够说明区域之间的出行联系强度，例如交通枢纽、商圈、居住区和办公区之间的流动关系。系统将 OD 结果预先汇总到 DWS 层，前端只需读取排名和流向数据即可展示。

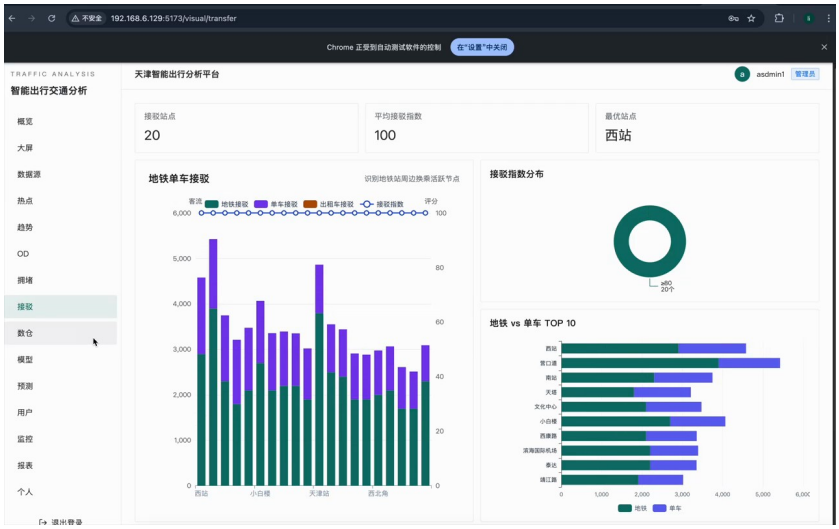

图 6-5 拥堵趋势与专题分析页面运行效果

图 6-5 展示了拥堵和专题分析结果。道路拥堵数据与天气、时间和道路等级相

关，页面通过趋势图、占比图和排行榜展示不同道路的拥堵程度。该页面可帮助用户从宏观趋势和局部道路两个角度理解交通状态。

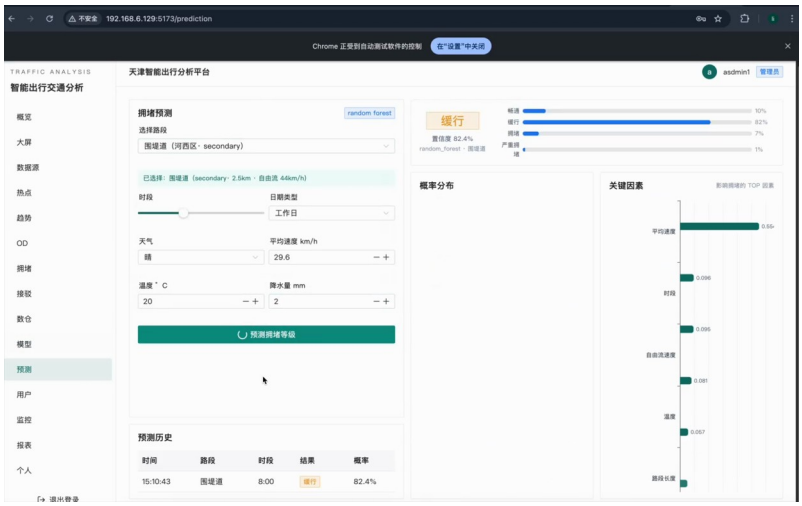

图 6-6 交通拥堵预测页面运行效果

图 6-6 展示了预测页面。用户可以输入道路、时间、天气等特征参数，系统调用模型接口返回预测结果。预测页面不仅展示预测值，还可显示风险等级和解释信息，使模型结果更便于普通用户理解。

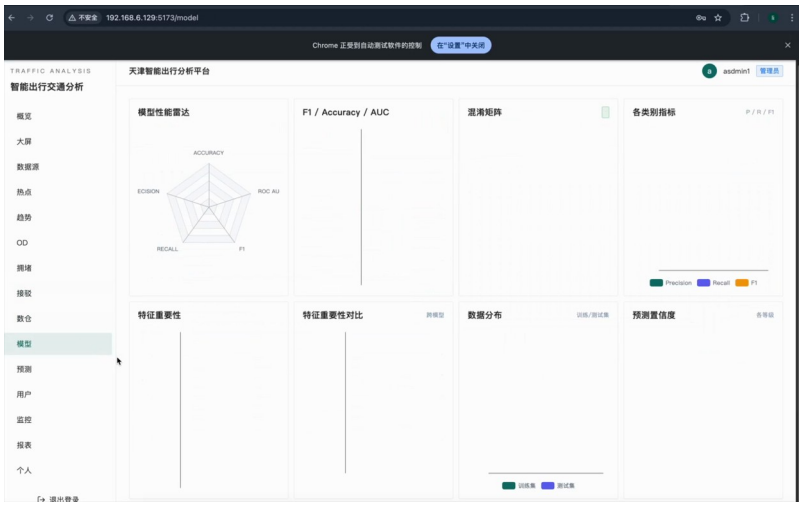

图 6-7 模型分析页面运行效果

图 6-7 展示了模型分析页面。该页面用于观察模型指标、训练状态和特征说明，

为预测模块提供结果解释支撑。

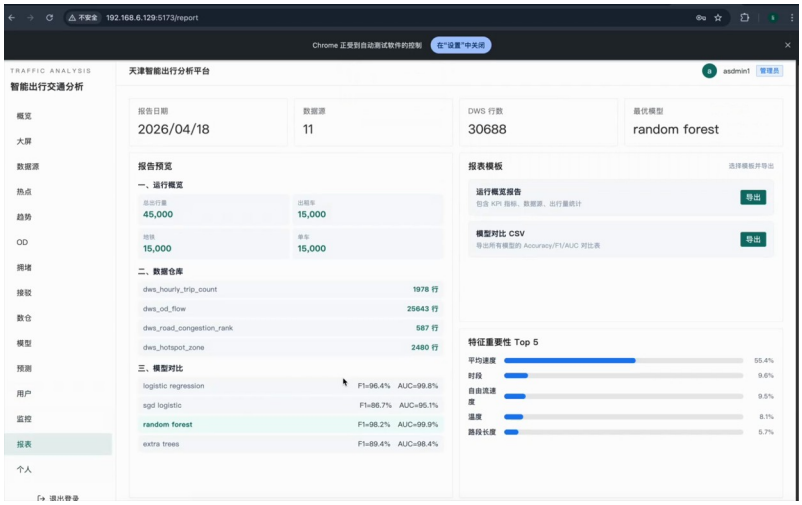

图 6-8 模型训练结果页面运行效果

图 6-8 展示了模型结果页面。页面包含模型指标、特征重要性和结果说明等内容，用于证明系统不仅完成静态可视化，也能够进一步基于 DWS 结果开展机器学习分析。

运行效果展示需要覆盖主要业务链路。首先展示登录或系统入口，其次展示数据源和任务页面，再展示综合大屏、趋势分析、OD 分析、拥堵分析和预测页面。这样的截图顺序能够说明系统从后台管理到前台分析均已实现。

运行截图不能只作为页面装饰，还要和正文说明对应。每张截图前应说明该页面的数据来源、展示内容和业务意义，截图后应解释页面运行结果。例如大屏截图展示了总出行量和热点分布，后文应说明这些指标来自 DWS 层聚合结果。

对于运行视频截取的图片，论文需要控制尺寸和清晰度。截图过大容易占满整

页，截图过小又看不清内容。本文在构建脚本中控制图片宽度，使页面截图与文字说明保持合理比例。

6.5 测试结果分析

从测试结果看，系统已形成较完整的“数据—计算—接口—展示”闭环。数据层能够提供多源交通样本，计算层能够完成清洗和聚合，后端能够输出统一 JSON 结构，前端能够展示综合大屏和专题分析结果，模型模块能够完成训练和预测展示。整体功能与任务书要求基本一致。

系统当前仍存在一定局限。第一，天津数据属于高真实性仿真数据，不能用于真实交通治理结论；第二，系统以离线批处理为主，尚未接入 Kafka 或 Spark Streaming 等实时流组件；第三，模型预测以传统机器学习为主，尚未引入复杂时空深度学习模型；第四，部分页面在大规模数据条件下还需要进一步优化分页、缓存和图表降采样。

这些局限不会影响毕业设计的核心目标，因为本文重点在于完成大数据系统设计与实现，而不是构建生产级城市交通平台。后续若要继续扩展，可接入更稳定的数据源、完善实时计算链路，并在模型侧引入时序预测和图神经网络等方法。

从运行截图可以看出，综合大屏和专题页面已经具备较强的展示完整性。地图、趋势、排名、模型输入和模型结果分布在不同页面中，避免了一个页面信息过载。对于论文写作而言，这些截图能够作为系统实现章节和测试章节的直接证据，说明项目确实存在可运行界面。

运行结果也反映出系统后续优化方向。例如数据源页面若处于加载或空状态，应进一步补充状态说明；模型页面可以增加训练时间、样本量和指标解释；大屏页面可以增加时间筛选与区域筛选。上述优化不影响当前系统核心功能，但可以作为展望部分的改进点。

测试结果表明，系统能够形成从数据到页面的闭环。数据源可登记，Spark 任务可产出结果，后端接口可读取结果，前端页面可展示图表，模型模块可返回预测结果。闭环完整性是本文系统实现最重要的评价点。

从数据质量结果看，生成数据能够体现早晚高峰、空间热点和天气扰动等规律，适合用于系统演示和论文分析。由于数据并非官方实测数据，本文不对天津真实交通运行状态作结论，只说明系统在构造数据上完成了分析流程验证。

从功能测试结果看，主要页面能够正常加载，接口返回结构稳定，任务状态能

够记录。部分功能仍有提升空间，例如任务调度可以加入异步队列，模型预测可以加入更多特征，地图页面可以支持更细粒度区域筛选。这些内容可作为后续改进方向。

从性能和可扩展性角度看，当前 5 万条主业务数据在本地环境可以较快完成处理，页面读取预聚合结果后响应较稳定。若未来接入更大规模数据，可以通过 HDFS 分区、Hive 列式存储、Spark 分布式执行和接口缓存进一步提升系统能力。

从测试覆盖情况看，系统已经覆盖数据层、接口层和页面层。数据层测试保证输入和聚合结果可用，接口层测试保证前端能获取稳定结构，页面层测试保证用户能看到分析结果。三层测试结合起来，比单独截图更能说明系统实现完整性。

从运行效果看，系统页面能够围绕交通主题形成连续展示。用户可以先在大屏看到总体态势，再进入趋势页面观察时间变化，进入 OD 页面观察空间流向，进入拥堵页面观察道路状态，最后进入模型页面查看预测结果。这种演示顺序与论文叙述顺序一致，有利于答辩讲解。

从不足看，当前测试更偏向功能验证和演示验证，压力测试和长期稳定性测试仍然不足。若系统用于真实业务场景，还需要模拟更大数据量、更高并发访问和更

长时间任务调度。毕业设计阶段以功能闭环为主，后续可在工程化部署中继续完善。

7 总结与展望

本文围绕天津智能出行交通分析场景，设计并实现了基于 Hadoop、Spark 和 Hive 的智能出行交通数据可视化分析系统。系统从多源交通数据出发，构建了原始数据存储、Hive 分层建模、Spark 离线计算、Django 接口服务、Vue 可视化展示和机器学习预测的完整流程，能够较好地体现数据科学与大数据技术专业的综合实践要求。

在系统设计方面，本文采用前后端分离架构和数仓分层思想，将系统划分为 Web 客户端、REST 接口、业务服务、大数据处理和存储层。ODS 层保存原始数据，DWD 层统一明细口径，DWS 层输出主题聚合结果，ADS 层面向接口和页面服务。该设计降低了页面查询与离线计算之间的耦合度，使系统具备较好的扩展性。

在系统实现方面，本文完成了交通数据源管理、Spark ETL 任务、小时趋势、OD 流向、热点区域、道路拥堵、模型训练和预测查询等模块。系统前端通过 ECharts 展示地图、折线图、柱状图、排行榜和模型结果，后端通过统一 API 向页面提供数据。结合实际运行视频截图可以看出，系统能够完成主要功能演示。

在测试与分析方面，系统对数据行数、坐标范围、峰谷特征、来源字段、天气

数据和真实性报告进行了验证，并对综合大屏、趋势分析、OD 分析、拥堵分析和预测页面进行了运行效果检查。测试结果表明，系统满足毕业设计中的功能展示和论文分析要求。

后续工作可以从三个方向展开。第一，接入更完整的真实公开交通数据或实时数据流，构建 Kafka、Spark Streaming 和实时大屏链路；第二，扩展路网拓扑、公交数据和停车数据，提高城市交通画像的完整性；第三，引入 LSTM、Transformer 或图神经网络等时空预测模型，提高拥堵预测和出行需求预测的精度。通过这些扩展，系统可以从毕业设计原型进一步发展为更完整的智能交通数据分析平台。

总体来看，本系统的实现路线具有较好的层次性。数据从生成、清洗到聚合逐步收敛，接口从元数据、任务到可视化逐步展开，前端从总览到专题逐步下钻。这样的组织方式有利于论文叙述，也有利于答辩时按照“为什么做、怎么设计、如何实现、结果如何”的逻辑进行说明。

同时，本文在数据口径上保持谨慎。系统使用的是高真实性仿真数据，适合展示系统能力和算法流程，但不把数值结论解释为真实城市交通运行结论。这种表述可以降低数据来源风险，也使论文结论更加严谨。

本文的不足主要体现在三个方面。第一，数据来源为高真实性仿真数据，虽然能够体现规律并支撑系统演示，但与真实城市交通平台仍有差距；第二，模型部分采用传统机器学习方法，尚未引入图神经网络、时空 Transformer 等更复杂模型；第三，系统主要以离线批处理为主，尚未接入 Kafka 和 Spark Streaming 等实时处理链路。

后续可以在数据接入方面进一步扩展。若能够接入公开交通流、公交客流、停车记录或真实道路速度数据，系统可以形成更完整的城市出行画像。同时，可以增加数据脱敏、隐私保护和权限审计机制，使系统更接近真实业务环境。

后续可以在算法方面进一步扩展。对于道路拥堵预测，可以引入 LSTM、GRU、时空图卷积网络或 Transformer 模型；对于热点区域分析，可以引入基于路网和 POI 的空间特征；对于 OD 流向预测，可以引入历史窗口和节假日特征。算法扩展需要建立在稳定数据链路之上，因此本文的数仓和接口设计仍具有基础价值。

后续可以在系统工程方面进一步扩展。任务调度可以接入 Celery 或 Airflow，接口层可以增加缓存和限流，前端可以增加更多筛选条件和报表导出功能，部署层可以使用 Docker Compose 统一服务。通过这些改进，系统可以从毕业设计原型逐步扩

展为更完整的交通数据分析平台。

通过本次毕业设计可以看出，大数据系统的难点并不只在某个算法，而在于数据链路是否稳定、字段口径是否统一、系统模块是否清晰。只要数据源、数仓、计算、接口和页面之间缺少任何一环，最终展示结果都难以令人信服。因此，本文把系统工程链路作为重点展开。

本文最终形成的论文和系统材料具有可继续维护的特点。正文内容、参考文献、图片和表格由脚本统一生成，后续若需要替换文献、增加表格或调整章节，可以从 JSON 和构建脚本入手。与纯手工编辑 Word 相比，这种方式更适合多轮修改，也更容易保持格式一致。

参考文献

[1] 靳向峰.基于大数据的智慧交通平台研究[J].科学与财富,2022:3(142-144).

[2] 汤柏龄.大数据技术在智慧交通的应用新模式[J].科学与信息化,2022:3.

[3] 邬群勇;王祥健.出租车轨迹数据的频繁轨迹识别[J].测绘通报,2021:6.

[4] 荆忠航;张伟;王佳慧;马利民;徐涛.面向 Hive 查询的存储优化技术[J].北京信息科技大学学报(自然科学版),2021:8.

[5] 面向轨迹大数据的轨迹分段方法研究及系统实现[D].北方工业大学,2021.

[6] 基于轨迹数据的快速地图更新[D].华东师范大学,2021.

[7] 基于 Hive 的电商数据分析平台的实现与应用[D].汕头大学,2021.

[8] 杨森;张蕊婷.高校数据中台建设研究[J].办公自动化,2022:4.

[9] 高雅麟;刘勇;贺含悦;任寅姿.智慧水务的数据中台建设[J].给水排水,2022:9.

[10] 邓娅茹.基于云服务的数据中台建设[J].信息与电脑(理论版),2023:3.

[11] 魏祥丽.智慧校园数据中台建设研究[J].现代计算机,2022:5.

[12] 卢小华;刘静.ECharts 技术在教师考评系统数据可视化中的应用[J].电子技术与软件工程,2022:4.

[13] 王蕾;王魁生.ECharts 在井眼轨迹可视化系统中的应用[J].现代计算机,2021:5.

[14] 林正敏,王淑娥,于小亿.基于 Django 的智慧养老系统设计[J].电脑知识与技

术,2023:4.

[15] 高焕超.基于 Django 的在线学习系统设计与实现[J].电脑编程技巧与维护,2021:3.

[16] 夏凡华.Go 前后端分离 Web 开发的实现[J].电脑编程技巧与维护,2021:4.

[17] 赵泽铖,汤运凯,匡仙丹.基于 SSM 的前后端分离式藏文学习系统设计[J].信息与电脑(理论版),2022:5.

[18] 孙鉴;刘凇佐;武晓晓;巫思敏.基于 Spark 的并行模拟退火算法求解 TSP[J].电子测量技术,2022:7.